

Banco de Dados II: Revisão

Sistema Gerenciador de Banco de Dados



Banco de Dados II

Jairo Lucas

Banco de Dados II: Revisão

■ Principais tópicos:

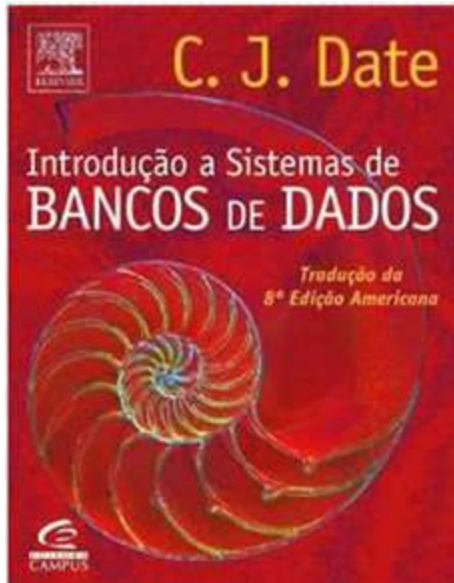
- Revisão (insert, update, group by, union, modelo conceitual, logico e fisico...)
- Consultas avançada em SQL,
- Programação em Sql
- Índices em Bancos de Dados,
- Views
- Stored Procedures
- Triggers
- Tratamento de erros (try / catch , Throw)
- Backup / Restore

Banco de Dados II: Revisão

■ Principais tópicos:

- Segurança em Bancos de Dados
- Aplicações Avançadas (Big Data, BI).

Introdução Banco de Dados: Bibliografia



Introdução a sistemas de bancos de dados. Tradução de Daniel Vieira. Rio de Janeiro: Campus,



Sistemas de Banco de Dados Elmasri Navathe – 4ª Edição

Apostilas , Slides e demais materiais dados em aula.

Banco de Dados II: Revisão

■ Introdução

Banco de Dados II: Introdução

- Por que estudar Banco de Dados II?
 - A famosa frase: “**Dados são o novo petróleo**”, se encaixa bem nesta disciplina. Assim como o petróleo, dados precisam ser extraídos, refinados e analisados para gerar valor.
 - Estamos na era do BIG DATA, na era **DIGITAL**.
 - Big Data **não substitui** o banco de dados, ele **depende dele**.

Banco de Dados II: Introdução

■ Por que estudar Banco de Dados II?

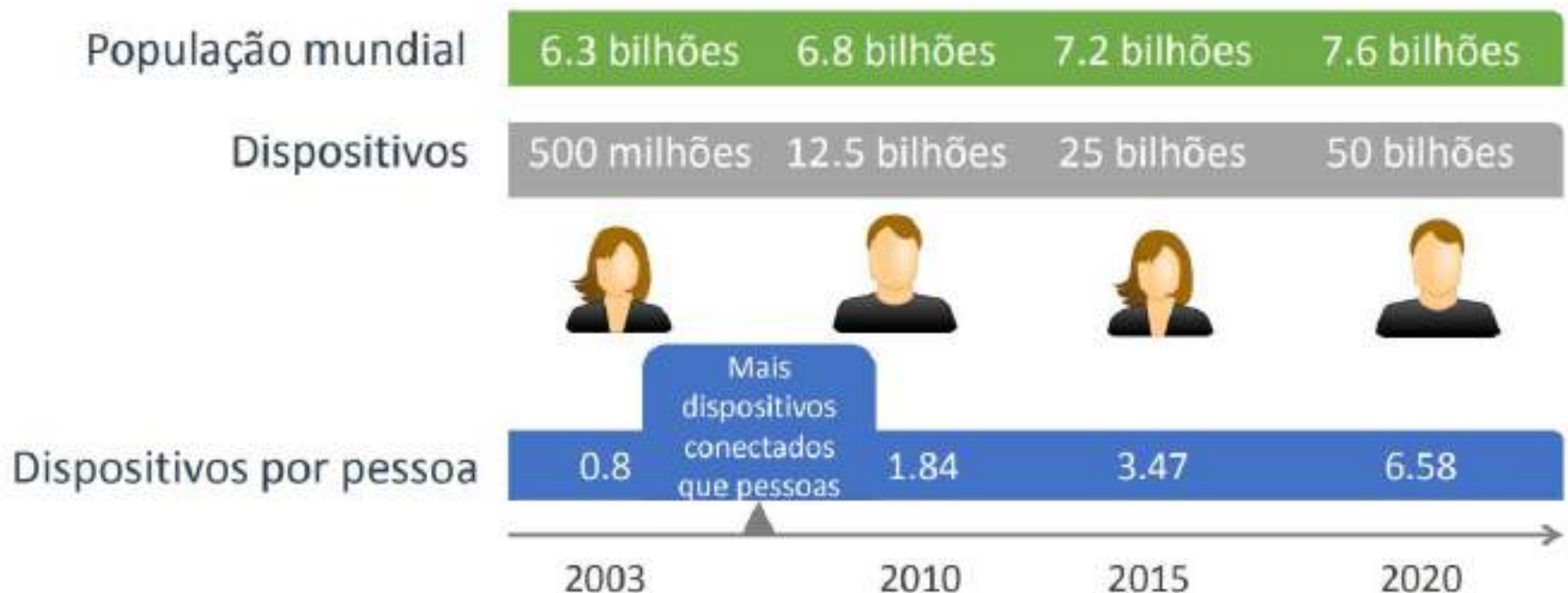
- **Dados gerados em redes sociais:** Textos, imagens, dados de áudio e vídeo em plataformas como WhatsApp, Facebook, Instagram e Twitter.
 - Facebook recebe cerca de 2 BILHÕES de acessos por dia
- **Streaming de filmes, séries e músicas:** Serviços como Netflix, Spotify e YouTube geram e processam uma enorme quantidade de dados diariamente.
- **Livros eletrônicos e jogos on-line:** A leitura de e-books e a participação em jogos on-line também contribuem para a produção de grandes volumes de dados.
- **Inteligência Artificial:** O desenvolvimento e a aplicação de IA geram grandes volumes de dados, que são utilizados para treinar algoritmos e melhorar sistemas inteligentes.

Banco de Dados II: Introdução

- Por que estudar Banco de Dados II?
- A **era digital** transformou a forma como a humanidade armazena e transmite informação, tornando os dados digitais não só predominantes, mas **quase onipresentes**, enquanto os dados impressos representam uma fração muito pequena desse total
- Estudo da IDC (International Data Corporation) mostra que o volume de dados digitais no mundo ultrapassou 120 zettabytes em 2023 (1 zettabyte = 1 trilhão de gigabytes).
- Já a produção de dados impressos (livros, jornais, revistas, documentos físicos) é ínfima comparada a isso.

Banco de Dados II: Introdução

■ Por que estudar Banco de Dados II?



Banco de Dados II: Introdução

- Por que estudar Banco de Dados II?
 - Segundo o **LinkedIn**, profissões ligadas a dados, como **Engenheiro de Dados**, **Cientista de Dados**, **Administrador de Banco de Dados (DBA)** e **Analista de BI** estão entre as **mais demandadas e bem pagas** do mercado de TI.
 - No Brasil, vagas para profissionais de dados cresceram mais de **485% nos últimos 5 anos** (Fonte: Catho/LinkedIn).
 - Entre os profissionais mais requisitados estão profissionais com domínio de SQL, modelagem de dados, performance de SGBDs e soluções em nuvem.

Banco de Dados II: Introdução

■ Por que estudar Banco de Dados II?

Cargo	Início de carreira (Júnior)	Profissional experiente (Pleno/Sênior)
Analista de Dados	R\$ 3.000 – R\$ 5.000	R\$ 7.000 – R\$ 12.000
Cientista de Dados	R\$ 4.900 – R\$ 6.500	R\$ 10.000 – R\$ 18.000+
DBA (Administrador de BD)	R\$ 3.500 – R\$ 5.500	R\$ 8.000 – R\$ 15.000

Banco de Dados II: Introdução

- Por que estudar Banco de Dados II?
- **Objetivos disciplina:**
 - escrever SQL mais rápido e mais correto
 - escrever consultas SQL complexas e eficientes
 - Criar índices e entender quando usá-los
 - diagnosticar consultas lentas
 - controlar transações, concorrência e tratamento de erros
 - automatizar regras de negócio no banco
 - entender quando usar trigger x sp x aplicação
 - Recuperar bancos quebrados
 - Administrar segurança, usuários e permissões

Banco de Dados II: Revisão

■ Banco de Dados

Banco de Dados II: Revisão

- “*Um **Banco de Dados** é uma coleção de dados inter-relacionados, representando informações sobre um domínio específico*”.
- Esses dados representam algum aspecto do mundo real (mini-mundo) e estão agrupados, logicamente, com significado implícito.
- Como exemplos de banco de dados, podemos citar:
 - Lista telefônica,
 - Controle do acervo de uma biblioteca,
 - Livro ponto uma empresa.

Banco de Dados II: Revisão

- **Sistema de Banco de Dados:** é formado por quatro componentes essenciais, que juntos permitem coletar, armazenar, processar e entregar informações de forma confiável e segura
 - Dados
 - Hardware
 - Software (o SGBD)
 - Usuários

Banco de Dados II: Revisão

■ Dados

- **Dados** de um SGBD é um conjunto de elementos dispersos, organizados segundo um modelo, de forma que, mesmo dispersos, os elementos possam ser integrados permitindo a extração de algum tipo de informação.
- Ou resumindo: São **elementos brutos**, como números, códigos, nomes, datas, valores, registros, etc.
- **Atenção!!** Dado $\leftrightarrow$ Informação

Banco de Dados II: Revisão

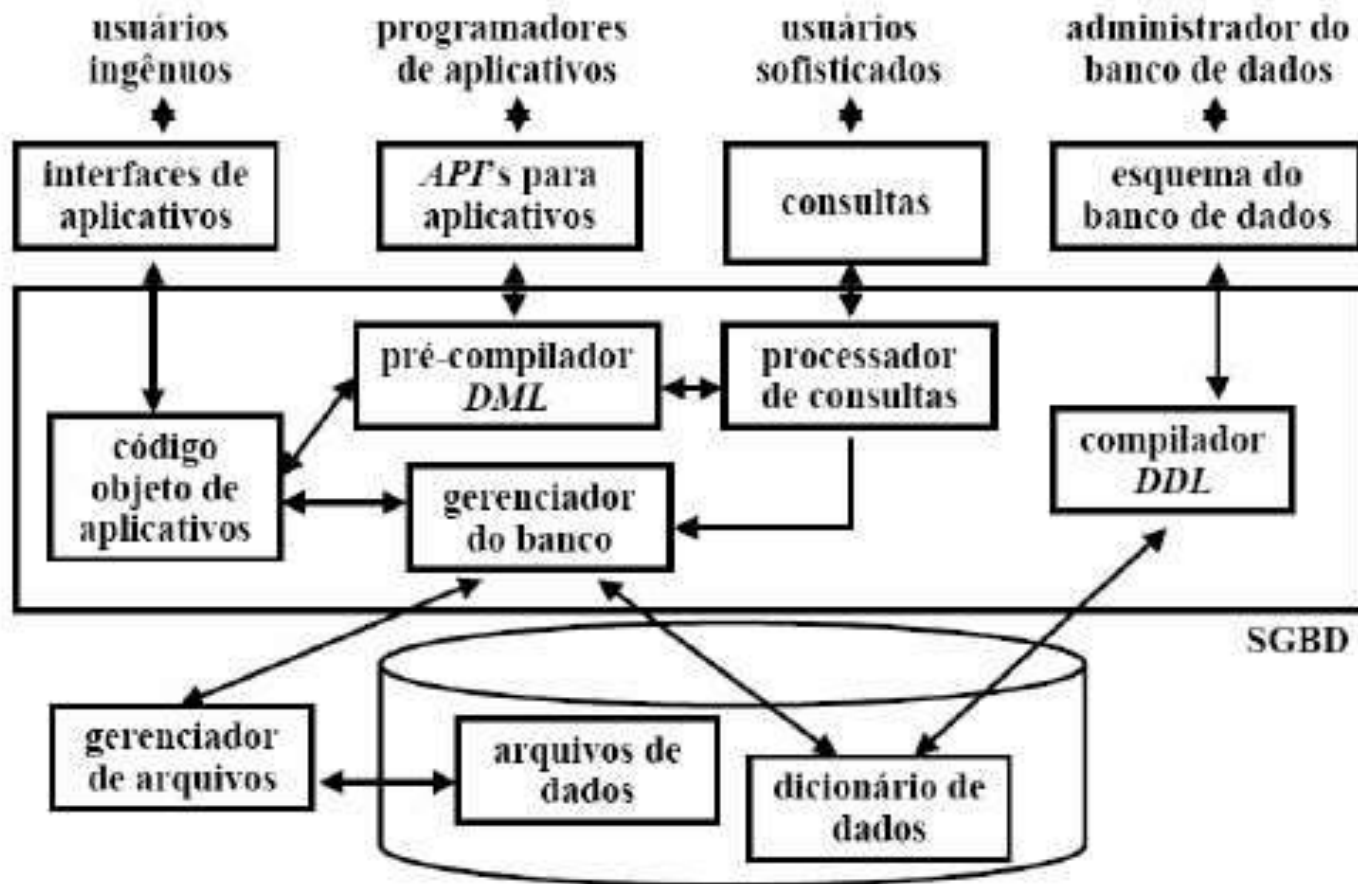
- **Dados** é apenas um índice, um registro, um número. **Normalmente, quando visto de forma isolada não tem muito significado.**
- **Informação** é um conjunto de dados que traz algum conhecimento/significado para o receptor. Basicamente, é a interpretação e entendimento de um conjunto de dados.
 - No contexto de SBD, a informação pode ser vista como o “produto final” da interação entre dados, software, hardware e usuário.

Banco de Dados II: Revisão

- **Hardware:** Toda a infraestrutura física responsável por armazenar e processar os dados.
- **Software (SGBD):** O sistema gerenciador de banco de dados (SGBD), responsável por organizar, manipular, proteger e permitir o acesso aos dados.
- **Usuários:** Pessoas ou aplicações que interagem com o banco de dados, utilizando ou manipulando informações.

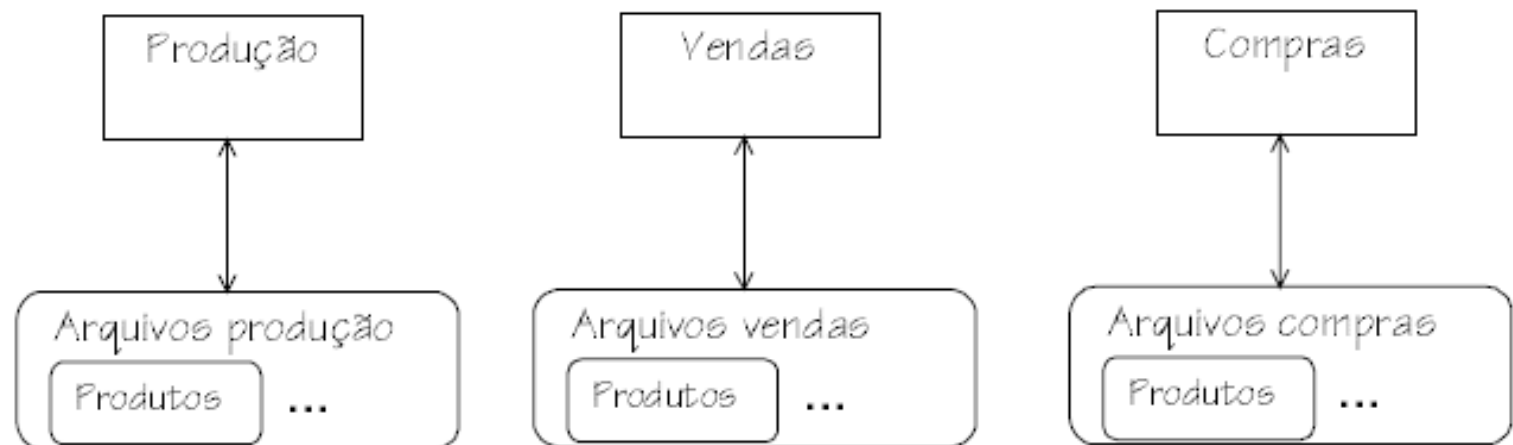
Banco de Dados II: Revisão

Estrutura Geral de um Sistema de Bancos de Dados



Banco de Dados II: Revisão

- Um SGBD deve prover algumas funções básicas:
 - **a) Controle de Redundância**



Banco de Dados II: Revisão

- Há duas formas de redundância de dados, a redundância *controlada* de dados e a redundância *não controlada* de dados.
 - A redundância *controlada* de dados acontece quando o software tem conhecimento da múltipla representação da informação e garante a sincronia entre as diversas representações. Essa forma de redundância é utilizada para melhorar a performance global do sistema.
 - Uma redundância *não controlada* de dados acontece quando a responsabilidade pela manutenção da sincronia entre as diversas representações de uma informação está com o usuário e não com o software. Este tipo de redundância geralmente é um erro, pois traz consigo vários tipos de problemas como inconsistência de dados e re-digitação.

Banco de Dados II: Revisão

- b) **Compartilhamento de dados:** O SGBD deve prover controle de concorrência para múltiplos usuários.
 - Em sistemas reais vários usuários acessam o banco ao mesmo tempo, e esses usuários podem tentar ler e alterar os mesmos dados. O SGBD é responsável por garantir que os dados permaneçam corretos, evitando conflitos entre acessos simultâneos

- c) **Restrições de Acesso a usuários:** Alguns usuários terão acesso a determinados recursos do banco enquanto outros não.
 - Um SGBD deve fornecer um subsistema de autorização e segurança que permita criar contas e especificar restrições nas contas. O SGBD deve então obrigar estas restrições automaticamente.

Banco de Dados II: Revisão

- d) **Fornecimento de Múltiplas Interfaces**: Um SGBD deve prover uma variedade de interfaces para atender usuários de com variado nível de conhecimento técnico.
 - Os tipos de interfaces incluem linguagens de consulta para usuários ocasionais, interfaces de linguagem de programação para programadores de aplicações, formulários e interfaces dirigidas por menus para usuários comuns;

- e) **Fornecer Restrições de Integridade**: A forma mais elementar de restrição de integridade é a especificação do tipo de dado de cada campo.
 - Existem tipos de restrições mais complexas. Por exemplo, a especificação de que um registro de um arquivo deve estar relacionado a registros de outros arquivos.

Projeto de Banco de Dados

- Um Projeto de banco de dados visa a organização das informações para que o futuro sistema obtenha boa performance.
- Normalmente seguem 3 fases:
 - Projeto Conceitual
 - Projeto Lógico
 - Físico
- Cada uma dessas fases tem objetivos específicos e contribui para a construção de uma base sólida para o armazenamento e manipulação de dados.

- **Projeto Conceitual**

- É composto pelo levantamento e análise de requisitos.
 - Entrevistas com usuários para entender suas necessidades
 - Após o levantamento de todos os requisitos é criado um Modelo Conceitual de Dados (MCD)
- **Modelo Conceitual de Dados:** É a descrição da estrutura do banco de dados
 - É **independente** do SGBD que será usado
 - Não incluem detalhes da implementação
 - Neste modelo, características e relacionamentos são representando independente das limitações impostas pela tecnologia.

- **Projeto Lógico**

- Nesta fase é efetuada a implementação do banco de dados utilizando um determinado SGBD

- No projeto lógico é definido o tipo de banco de dados a ser usado (relacional, hierárquico...).
- O resultado desta fase é um Modelo Lógico de Dados (MLD).

- **Um Modelo Lógico de Dados (MLD):** É a descrição de um banco de dados em função do modelo de SGBD adotado.

- Neste modelo, características, relacionamentos e objetos do banco de dados devem levar em consideração as limitações do SGBD adotado .
- Deve observar os conceitos de normalização, integridade referencial, integridade de domínio etc.

Banco de Dados II : Revisão

- **Projeto Conceitual**
 - Entidades
 - Atributos
 - Relacionamentos
- No projeto conceitual temos o **MER (Modelo Entidade Relacionamento)**, que é um modelo amplamente usado para representar um projeto conceitual.

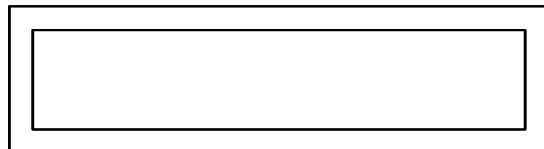
Banco de Dados II : Revisão

- **Projeto Conceitual – MER**

- **Entidades** – É o objeto básico do MER onde são armazenados as informações. Uma entidade é formada por atributos e valores.
- No projeto lógico de um banco de dados relacional como o SQL Server, as entidades são representadas como **TABELAS**.



Tipo de Entidade



Tipo de entidade fraca

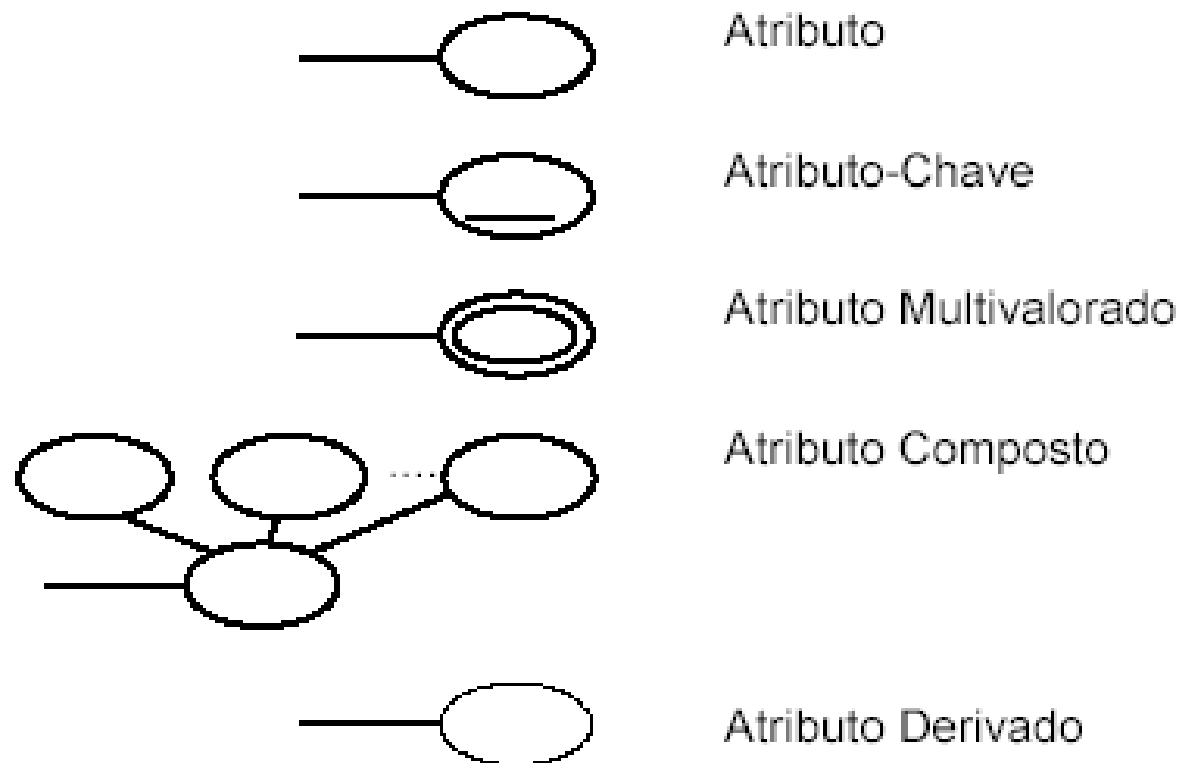
Banco de Dados II : Revisão

- **Projeto Conceitual – MER**

- **Atributos** – São as “partes” que formam uma entidade.
- No projeto lógico de um banco de dados relacional como o SQL Server, os **ATRIBUTOS** são representadas pelos **CAMPOS (colunas)** das tabelas.

Banco de Dados II : Revisão

- **Projeto Conceitual – MER**
 - **Atributos**



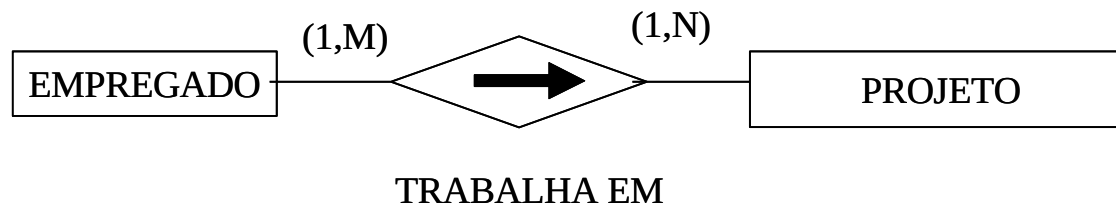
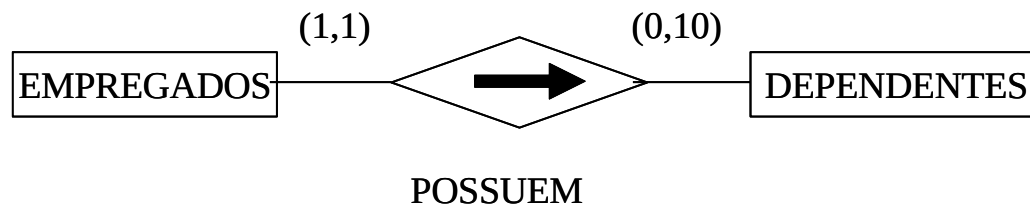
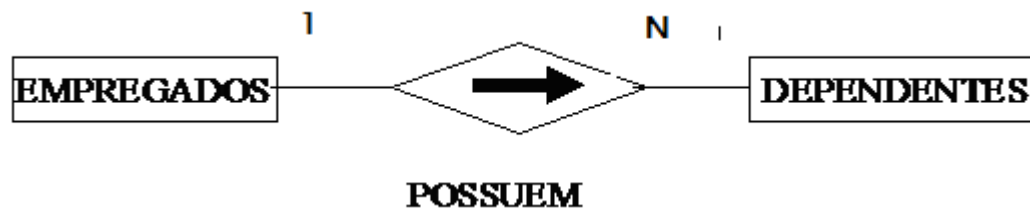
Banco de Dados II : Revisão

- **Projeto Conceitual – MER**

- **Relacionamento** : É uma estrutura que indica associações entre duas ou mais entidades.
 - Exemplo: Um relacionamento 'trabalha para' entre as entidades 'empregado' e 'departamento'
- No projeto lógico de um banco de dados relacional como o SQL Server, os **RELACIONAMENTOS** podem ser representados por **ligações entre as tabelas** (chaves públicas e privadas) ou através de **novas TABELAS**.

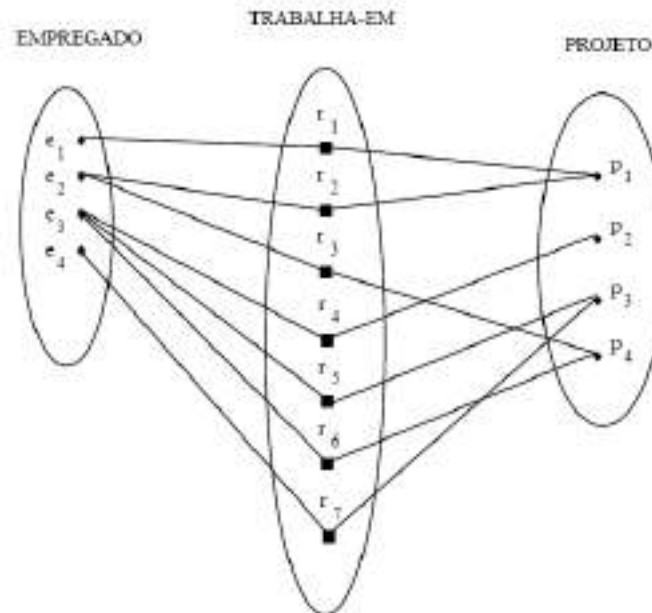
Banco de Dados II : Revisão

- **Projeto Conceitual – MER**
 - **Relacionamento**

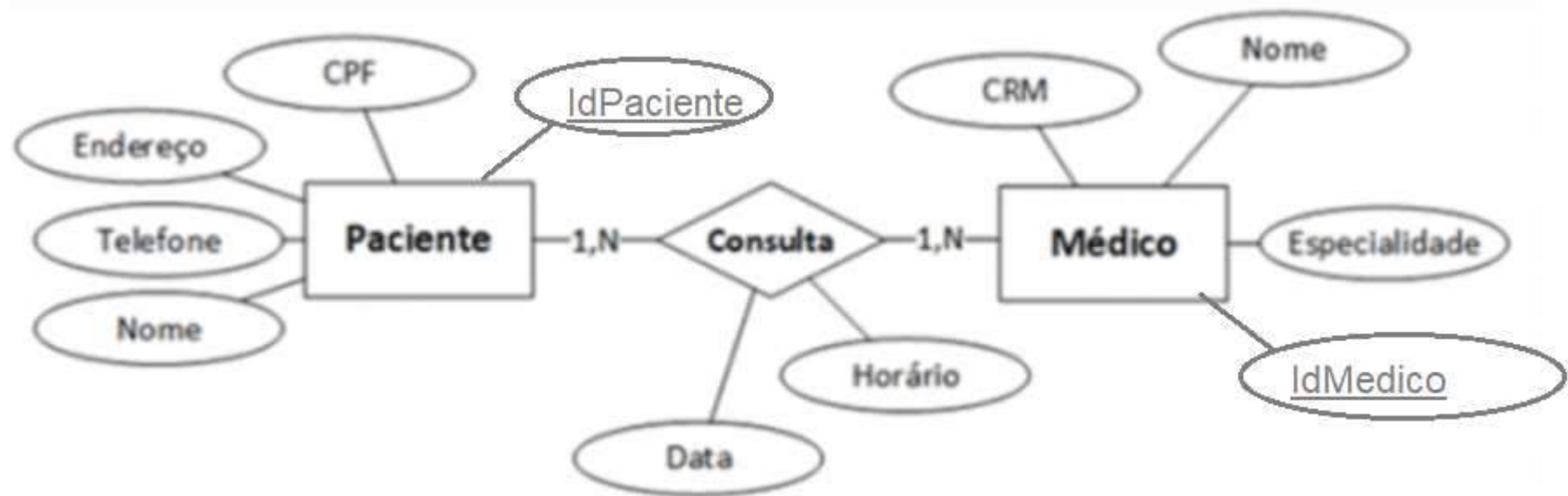


- **Conceitos (MER) - Relacionamentos**
 - **Grau de um relacionamento:** Define o número de entidades que participam de um relacionamento.
 - **Cardinalidade de um relacionamento:** Qtd de instâncias que um relacionamento pode participar.
 - Exemplo: No relacionamento 'Trabalha para' entre as entidades empregado e departamento possui cardinalidade 1:1, ou seja, o um empregado pode trabalhar em apenas um setor
 - No relacionamento 'Possue ' entre as entidades departamento e empregado possui cardinalidade 1:N, ou seja, um departamento pode possuir vários empregados.

- **Conceitos (MER) - Relacionamentos**
 - **Cardinalidade de um relacionamento:**
 - Exemplo: O tipo de relacionamento TRABALHA-EM entre EMPREGADO e PROJETO tem a razão de cardinalidade M:N considerando que um empregado pode trabalhar em diversos projetos e que diversos empregados podem trabalhar em um projeto.



Banco de Dados II: Revisão



SGBD Relacionais

Modelagem de Dados

- 1) Quais as fases do projeto de banco de dados? Explique a finalidade de cada uma.
- 2) Por que o projeto conceitual deve ser independente do SGBD que será utilizado na implementação?
- 3) Considere um sistema de controle acadêmico. Explique quando um elemento do mundo real deve ser modelado como entidade e quando ele deve ser apenas um atributo.
- 4) Explique o conceito de cardinalidade em um relacionamento. Por que a definição correta da cardinalidade é importante no projeto de um banco de dados?
- 5) Quando um relacionamento do MER deve virar uma tabela (entidade associativa) no modelo lógico e quando ele pode ser representado apenas por uma chave estrangeira (FK)? Explique e dê um exemplo de cada caso.

SGBD Relacionais

Modelagem de Dados

- **Projeto Lógico - Modelo LÓGICO de Dados**
 - O modelo lógico é uma tradução do modelo conceitual para uma determinada tecnologia, ou seja, para um SGBD de determinado modelo.
 - Esta tradução é feita principalmente sobre as entidades, atributos e estrutura de relacionamentos.
 - Esta conversão segue um conjunto de regras que depende da tecnologia escolhida (modelo relacional, modelo orientado a objeto, modelo em rede...)
 - Aplicaremos a conversão para o modelo relacional.

SGBD Relacionais

Modelagem de Dados

- **Projeto Lógico -**

- **TABELAS** : No modelo lógico, as entidades viram TABELAS.
 - As tabelas são formadas por linhas e colunas
 - O número de linha é a cardinalidade da tabelas, o número de coluna o grau.
 - Cada elemento está em uma linha, e cada item de dado em uma coluna.

Paciente	
🔑	idpaciente
	cpf
	Endereco
	Telefone
	nome

idpaciente	cpf	Endereco	Telefone	nome
1	111.222.333-55	vila velha	333	carlos
2	222.123.434-56	Vitória	9876-3434	José Maria
3	333.321.345-98	Vitória	887-0987	Jairo Lucas
4	456.445.876-45	Serra	876-9839	Antonio

SGBD Relacionais

Modelagem de Dados

- **Projeto Lógico**

- **TABELAS** : No projeto lógico, uma tabela NORMALIZADA **somente pode existir** se respeitar algumas propriedades:
 - Cada campo pode ser vazio ou conter no máximo 1 valor.
 - A ordem das linhas é irrelevante do ponto de vista do usuário.
 - Não há duas linhas iguais ???!!!
 - Cada coluna tem um nome
 - Duas colunas distintas devem ter nomes distintos.
 - A partir do nome, a ordem das colunas é irrelevante.
 - Cada tabela tem um nome próprio diferente dos demais nomes de tabelas.
 - Os valores de uma coluna são retirados de um conjunto denominado “Domínio”.

SGBD Relacionais

Modelagem de Dados

- **Projeto Lógico**

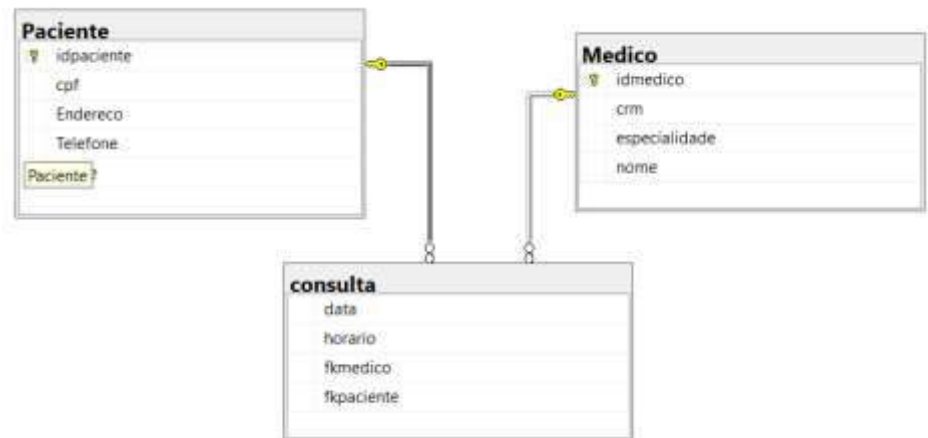
- **Ligações** : Relacionam duas ou mais tabelas. São equivalentes aos **relacionamentos** do modelo conceitual
 - Duas tabelas (T1) e (T2) possuem ligações através das colunas C1 de T1 e C2 de T2 se, e somente se:
 - C1 e C2 tem o mesmo domínio
 - C1 e C2 possuem o mesmo valor em determinado instante.
 - A ligação é feita transpondo a chave primária de uma tabela (PK) para outra tabela.(FK)

SGBD Relacionais

Modelagem de Dados

- **Projeto Lógico**

- **Ligações** : Relacionam duas ou mais tabelas. São equivalentes aos **relacionamentos** do modelo conceitual



idmedico	crm	especialidade	nome	cod
1	2233-RS	pediatra	Jose Luis	1
2	883738-ES	Pediatra	Paulo	1
3	87309-RS	Geriatra	Julio Cesar	2
4	8737-9873	Cardiologista	Renato Moura	3

Messages		
idpaciente	cpf	Endereco
1	111.222.333-55	vila velha
2	222.123.434-56	Vitória
3	333.321.345-98	Vitória
4	456.445.876-45	Serra

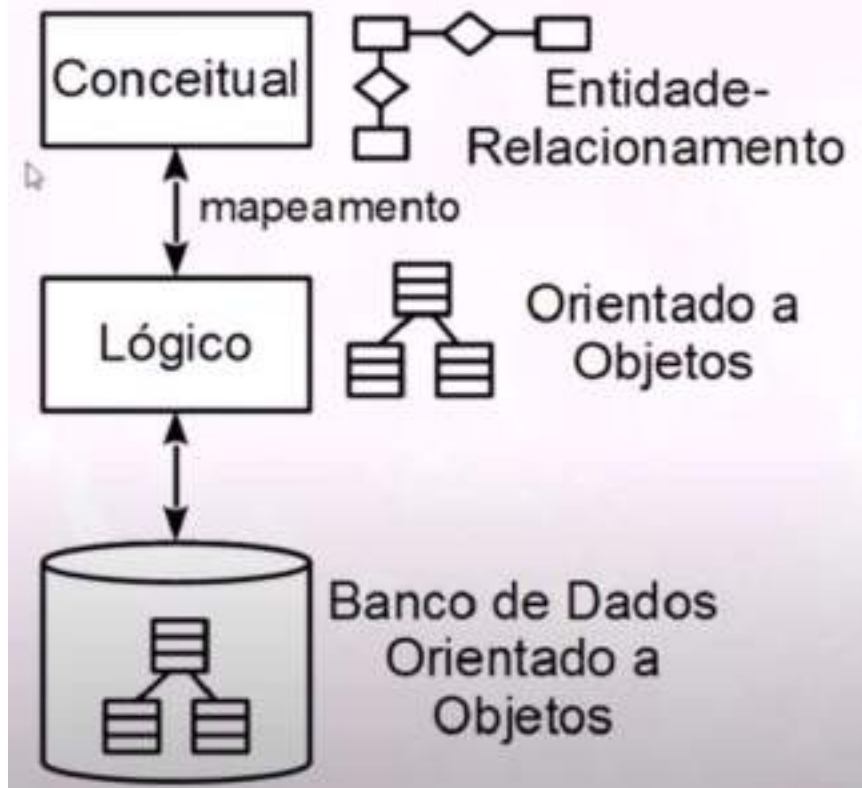
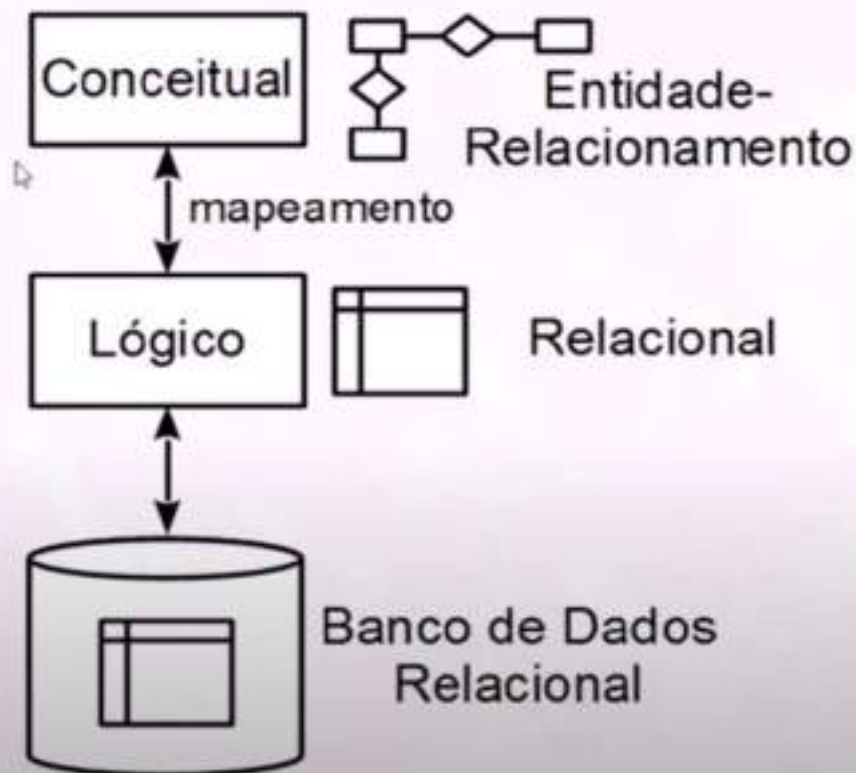
data	horario	fkmedico	fkpaciente
2024-05-03	10:15	1	1
2024-05-03	20:20	1	1
2024-05-03	15:24	3	4

- **Projeto Físico**

- Nesta fase é definido as estruturas de armazenamento interno, índices, caminhos, e organização dos arquivos.
 - O resultado desta fase é um Modelo Físico de Dados (MFD).
- **Um Modelo Físico de Dados (MFD):** É a implementação física do banco de dados de acordo com o SGBD escolhido.
 - A representação do objeto é feita do ponto de vista do nível físico

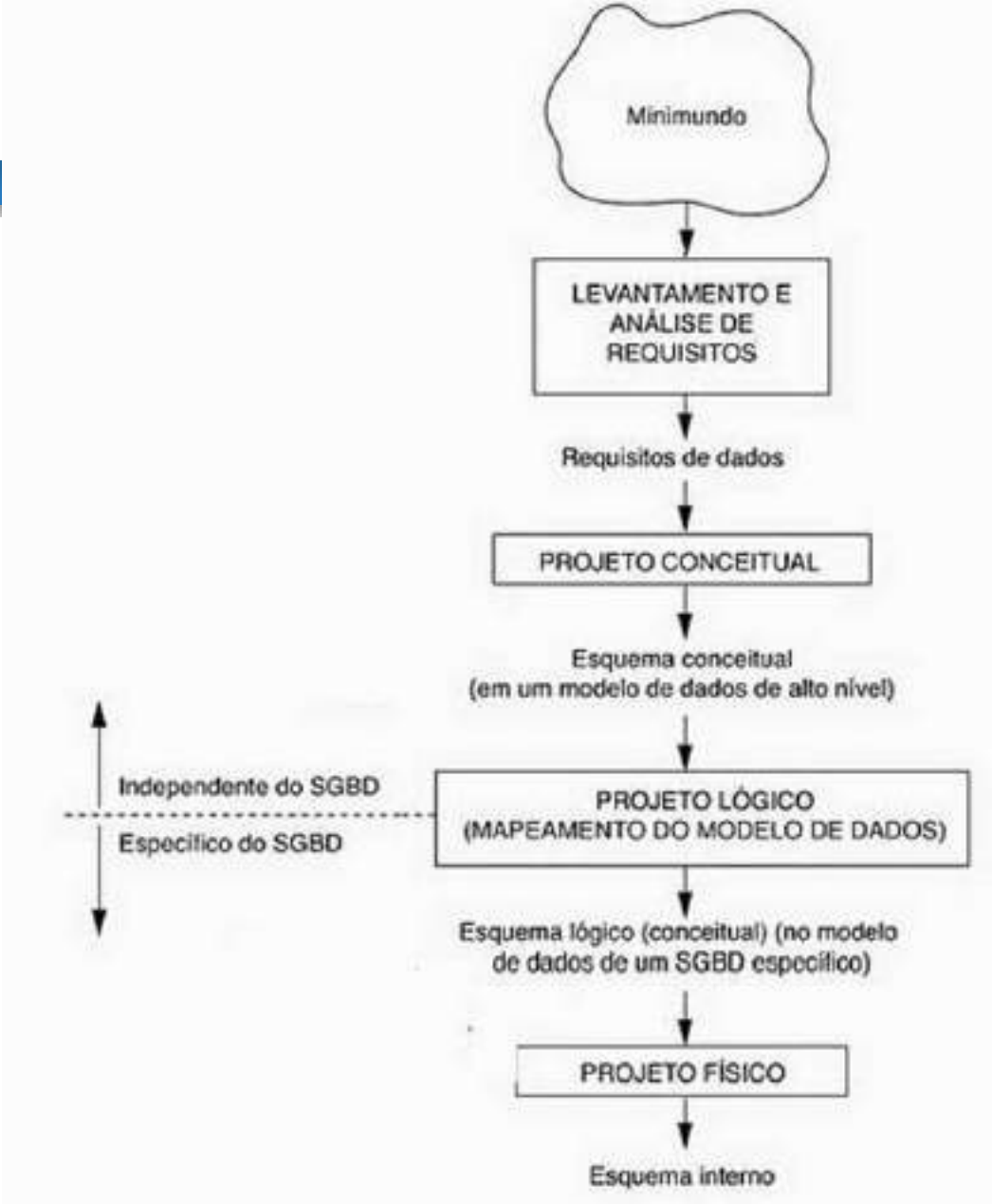
Banco de Dados II: Revisão

Computação Convencional	Matemática	Banco de dados (Modelo Lógico)	Banco de dados Modelo Coneitual
Arquivo	Relação	Tabela	Entidade
Campo	Atributo	Coluna	Atributo
Registro	Tupla	Linha	Valores

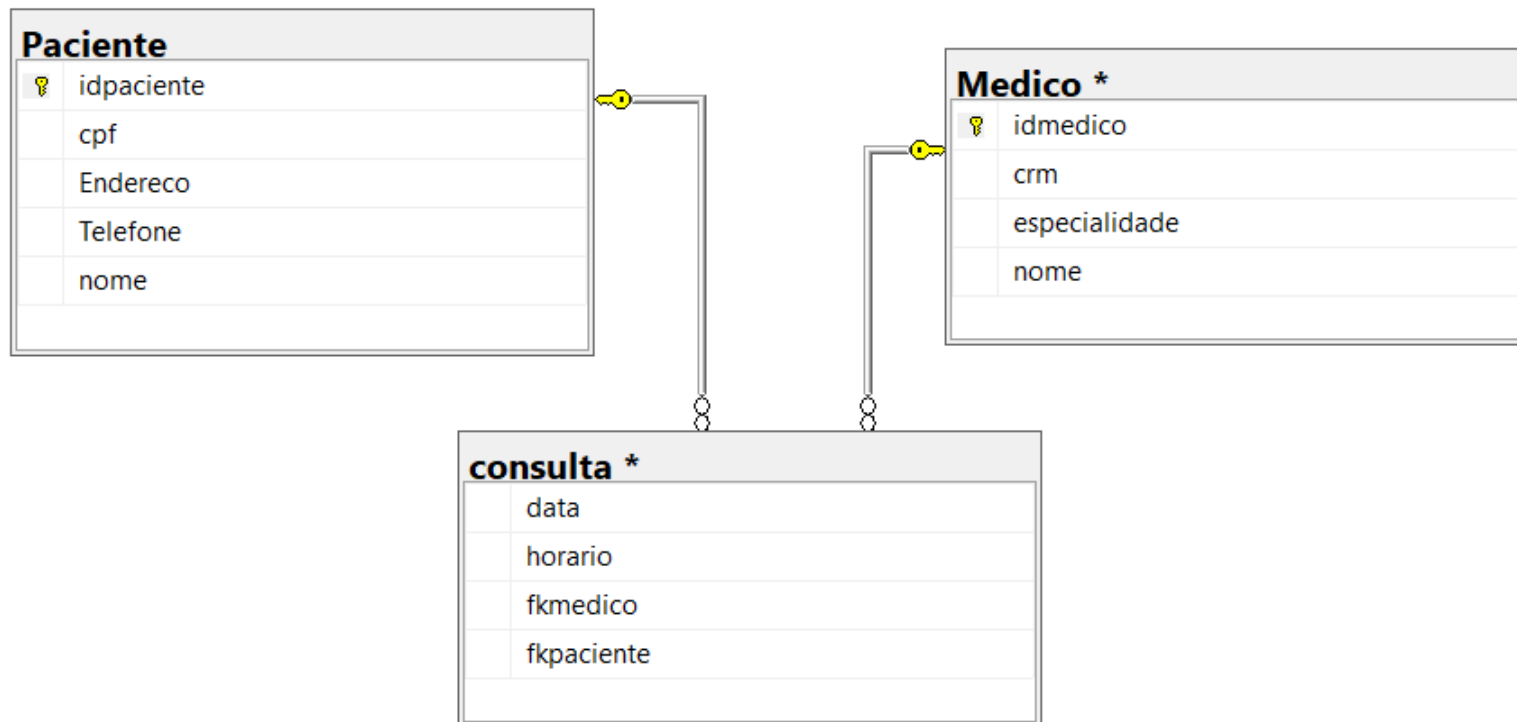




Computação Convencional	Matemática	Banco de dados (Modelo Lógico)	Banco de dados Modelo Coneitual
Arquivo	Relação	Tabela	Entidade
Campo	Atributo	Coluna	Atributo
Registro	Tupla	Linha	Valores



Banco de Dados II: Revisão



A arquitetura “Three-Schema”:

A arquitetura mais difundida na literatura é a Arquitetura “Three-Schema”. Nesta arquitetura, esquemas podem ser definidos em três níveis:

- **O nível interno:** Descreve a estrutura de armazenamento físico da base de dados. O esquema interno usa um modelo de dados físico e descreve todos os detalhes de armazenamento de dados e caminhos de acesso à base de dados;
- **O nível conceitual:** Descreve a estrutura de toda a base de dados. O esquema conceitual é uma descrição global da base de dados, que omite detalhes da estrutura de armazenamento físico e se concentra na descrição de entidades, tipos de dados, relacionamentos e restrições.
- **O nível externo ou visão:** Cada esquema externo descreve a visão da base de dados de um grupo de usuários da base de dados.

Banco de Dados II: Revisão

- **Em um banco de dados poderá haver:**
 - Muitas “visões externas” distintas.
 - Apenas uma “visão conceitual”, consistindo em uma representação abstrata do banco de dados em sua totalidade.
 - apenas uma “visão interna” representando o modo como o banco de dados será fisicamente armazenado.
- A maioria dos SGBDs não separa os três níveis completamente, mas suporta a arquitetura de três esquemas
- Os três esquemas são apenas descrições dos dados; na verdade, o dado que existe de fato está no nível físico.
- Os processos de transformação entre os níveis são chamados **mapeamentos.**

Banco de Dados II: Revisão

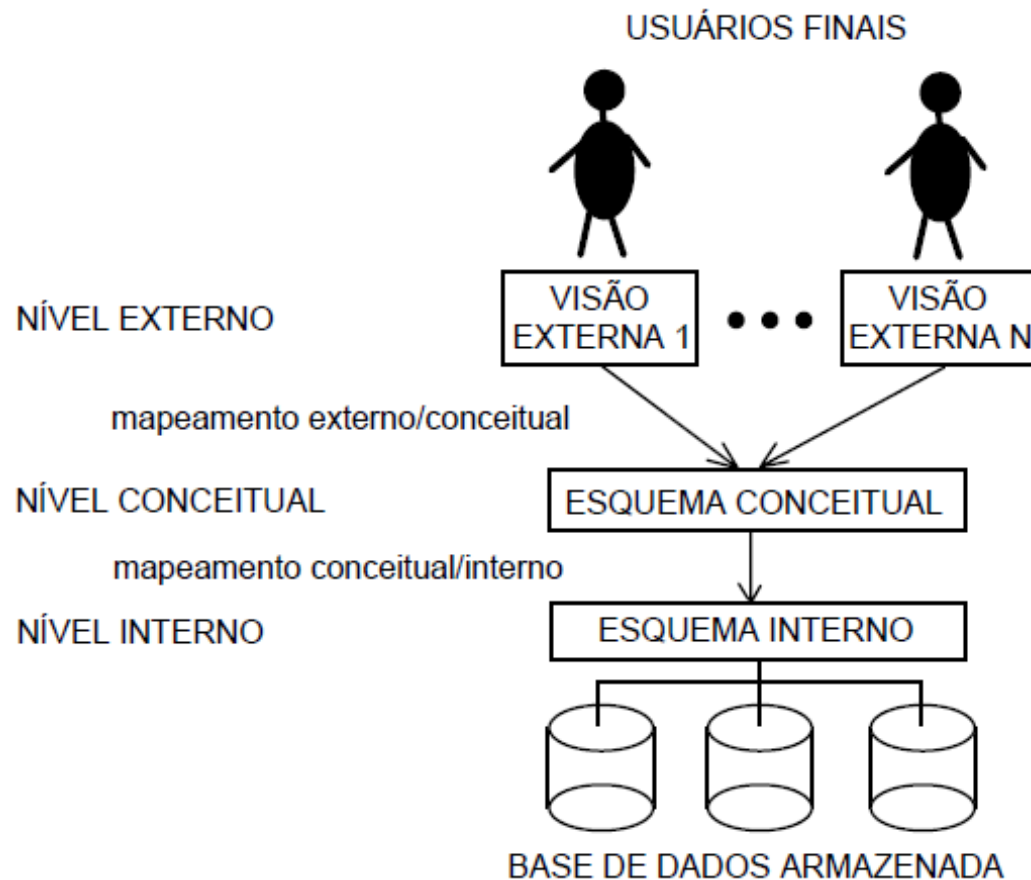


Figura 3.1 – Arquitetura Three-Schema

- **Independência de dados:** É a capacidade de mudar o esquema em um nível do sistema de banco de dados sem que ocorram alterações do esquema no próximo nível mais alto. É possível definir, ainda, dois tipos de independência de dados:
 - **Independência de dados lógica:** é a capacidade de alterar o esquema conceitual sem mudar o esquema externo ou os programas.
 - Exemplo : Criação de um novo campo
 - **Independência física de dados:** refere-se à capacidade de mudar o esquema interno sem ter de alterar o esquema conceitual.
 - Exemplo: Mudança do path do arquivo

Introdução a SGBD Relacionais

Banco de Dados II: Revisão

- **Linguagem SQL:** O SQL serve como a linguagem padrão para comunicar-se com a maioria dos SGBDs.
 - **Independência do SGBD:** Embora diferentes SGBDs possam implementar o SQL com algumas variações, a essência da linguagem SQL permanece a mesma.
 - Isso proporciona uma certa medida de independência do SGBD, permitindo que as habilidades em SQL sejam transferíveis entre diferentes sistemas de banco de dados

Banco de Dados II: Revisão

- A linguagem SQL tem três divisões principais:
 - **DDL: Linguagem de Definição de Dados (*Data Definition Language*):**
 - Domínios
 - Criação de Tabelas
 - Restrições de Integridade
 - Alteração de Tabelas
 - Eliminação de Tabelas
 - **DML: Linguagem de Manipulação de Dados (*Data Manipulation Language*):**
 - Inclusão de dados
 - Alterações / exclusão de dados
 - Consultas

Banco de Dados II: Revisão

- A linguagem SQL tem três divisões principais:
 - **DCL (Data Control Language)** - Linguagem de Controle de Dados
 - usada para definir controle de acesso a dados em um banco de dados.
 - GRANT: Concede privilégios de acesso a usuários e grupos.
 - REVOKE: Remove privilégios de acesso anteriormente concedidos.

Dependendo da fonte, podem existir ainda outras divisões como:

- **TCL (Transaction Control Language)** - Linguagem de Controle de Transações
- **DQL (Data Query Language)** - Linguagem de Consulta de Dados
 - Embora tecnicamente o SELECT seja frequentemente considerado parte da DML, algumas fontes se referem à DQL como uma categoria à parte focada especificamente na consulta de dados

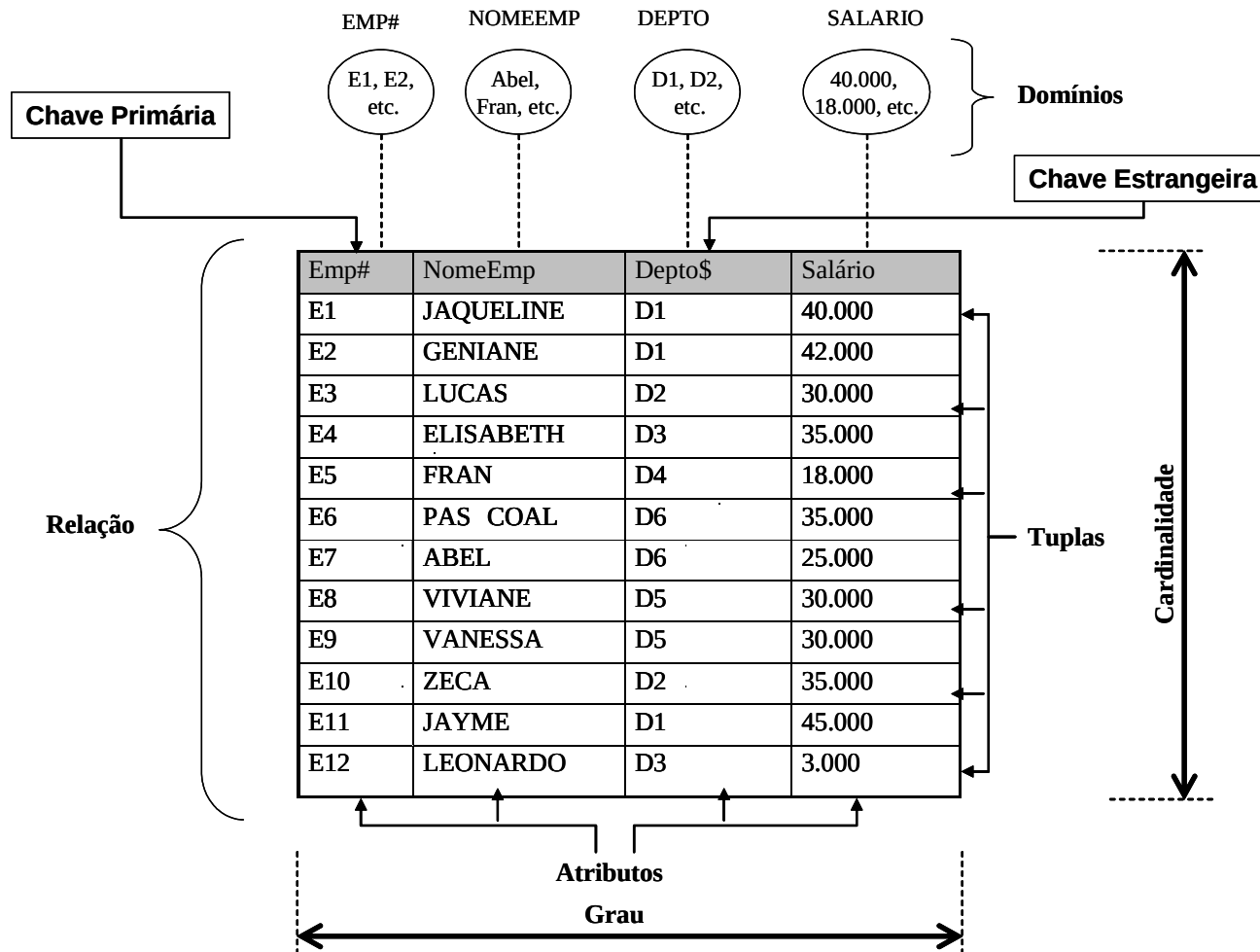
Banco de Dados II: Revisão

- O Modelo de Dados Relacional foi introduzido por Ted Codd (1970).
- É o mais simples, mais formal e largamente o mais usado
- É baseado em relações matemáticas e na teoria dos conjuntos
- O modelo Relacional representa o banco de dados como uma coleção de relações ou mais informalmente, um coleção de tabelas.

- Formalmente utiliza conceitos como:
 - Tupla : Representa uma linha da relação (tabela)
 - Atributo : Representa uma coluna da relação (tabela)
 - Relação : Representa a tabela

Computação Convencional	Matemática	Banco de dados (Modelo Lógico)	Banco de dados Modelo Coneitual
Arquivo	Relação	Tabela	Entidade
Campo	Atributo	Coluna	Atributo
Registro	Tupla	Linha	Valores

Banco de Dados II: Revisão



Aspectos Integridade (3)

- **Aspectos Integridade (3)**

- As restrições de integridade fornecem meios para garantir a integridade dos dados
- As restrições de integridade resguardam o banco de dados contra danos acidentais.
- A integridade no modelo relacional compõe-se das seguintes restrições:
 - *Integridade da Entidade.*
 - *Integridade Referencial*
 - *Integridade de Domínio.*

- **Aspectos Integridade (3)**
 - **Chave candidata e chave primária:** Como vimos, uma das regras do modelo relacional é não haver elementos repetidos. Para cumprir esta regra surge a figura da **chave primária**.
 - Uma **chave candidata** é um ou mais atributos que nos permite identificar de forma inequívoca um determinado elemento da relação.
 - A **chave primaria** é a chave candidata que o projetista escolheu como chave identificadora.

● Aspectos Integridade (3)

■ Chave candidata e chave primária:

Messages				
idpaciente	cpf	Endereco	Telefone	nome
1	111.222.333-55	vila velha	333	carlos
2	222.123.434-56	Vitória	9876-3434	José Maria
3	333.321.345-98	Vitória	887-0987	Jairo Lucas
4	456.445.876-45	Serra	876-9839	Antonio

■ As chaves candidatas são:

- Id,
- CPF
- Telefone

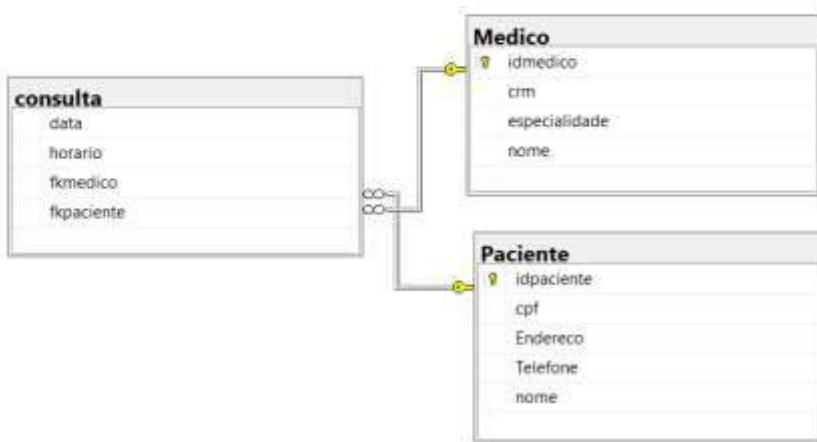
■ Chave Primária escolhida:

- ????

■ A garantia da unicidade da chave primária deve ser garantida pelo SGBD. Opcional pode-se fazer isso através da própria aplicação, **o que é uma péssima opção.**

Banco de Dados II: Revisão

- **Chave estrangeira (Foreign Key):** É uma chave usada para relacionar duas tabelas. Na prática, é uma chave primária que é repetida em outra tabela para possibilitar uma ligação entre as tabelas.



idmedico	crm	especialidade	nome	cod
1	2233-RS	pediatra	Jose Luis	1
2	883738-ES	Pediatra	Paulo	1
3	87309-RS	Geriatra	Julio Cesar	2
4	8737-9873	Cardiologista	Renato Moura	3

data	horario	fkmedico	fkpaciente
2024-05-03	10:15	1	1
2024-05-03	20:20	1	1
2024-05-03	15:24	3	4

idpaciente	cpf	Endereco
1	111.222.333-55	vila velha
2	222.123.434-56	Vitória
3	333.321.345-98	Vitória
4	456.445.876-45	Serra

- **Aspectos Integridade (3)**

- *Integridade da Entidade*: É garantida através da chave primária.
 - A chave primaria (PK) é um identificador ÚNICO que permite identificar uma determinada linha em uma tabela.
 - Cada tabela poderá ter apenas UMA chave primária
 - Não pode ser nula
 - Preferencialmente, deve-se usar chaves primárias numéricas (números inteiros), porém, isso NÃO é obrigatório para o banco de dados.

- **Aspectos Integridade (3)**

- **Integridade Referencial:** Garante que quando uma entidade (tabela) 1 esta relacionada com uma tabela 2 ,através de uma chave estrangeira, o valor desta chave estrangeira deve existir na chave primária da tabela 2.
 - Exemplo: A tabela Dependente esta relacionada com a tabela Funcionário através do campo Fkfuncionário. Para gravar um registro na tabela dependente com o valor 5 no campo Fkfuncionário, deve existir um funcionário com o código 5.
 - **Pode-se excluir o alvo de uma chave estrangeira?** (no exemplo acima, excluir o código 5 da tabela funcionário).

- *Integridade Referencial (continuação...)*
 - *Três formas de tratar a exclusão do alvo de uma chave estrangeira. (vale também para alteração):*
 - *Cascades* : Deleção em cascata. Neste caso, ao se excluir o alvo de uma chave estrangeira, é excluído também todos os registros que “apontam” para esta chave estrangeira. No exemplo anterior, seriam excluídos TODOS os dependentes que apontavam para o código 5.
 - *Restricted* : Não aceita excluir registros que são alvos de chaves estrangeiras.
 - *Nullifies* : Todas os registros que apontam para a chave excluída são ajustados para NULL.

- **Aspectos Integridade (3)**
 - *Integridade de Domínio:* Um domínio é um conjunto de valores, todos do mesmo tipo, logo, a integridade de domínio deve garantir que um atributo (campo) aceite somente dados dentro do seu domínio.

Projeto de Banco de Dados - Normalização

- **Projeto Lógico -**

- **TABELAS** : No modelo lógico, as entidades viram TABELAS.

- As tabelas são formadas por linhas e colunas
- O número de linha é a cardinalidade da tabelas, o número de coluna o grau.
- Cada elemento está em uma linha, e cada item de dado em uma coluna.

Paciente	
🔑	idpaciente
	cpf
	Endereco
	Telefone
	nome

idpaciente	cpf	Endereco	Telefone	nome
1	111.222.333-55	vila velha	333	carlos
2	222.123.434-56	Vitória	9876-3434	José Maria
3	333.321.345-98	Vitória	887-0987	Jairo Lucas
4	456.445.876-45	Serra	876-9839	Antonio

- **Projeto Lógico**

- **TABELAS** : No projeto lógico, uma tabela NORMALIZADA **somente pode existir** se respeitar algumas propriedades:
 - Cada campo pode ser vazio ou conter no máximo 1 valor.
 - A ordem das linhas é irrelevante do ponto de vista do usuário.
 - Não há duas linhas iguais ???!!!
 - Cada coluna tem um nome
 - Duas colunas distintas devem ter nomes distintos.
 - A partir do nome, a ordem das colunas é irrelevante.
 - Cada tabela tem um nome próprio diferente dos demais nomes de tabelas.
 - Os valores de uma coluna são retirados de um conjunto denominado “Domínio”.

- **Projeto Lógico**

- **Normalização da estrutura de Dados:** A normalização é processo formal aplicado em um banco de dados a fim de implementar regras para uma boa organização dos dados, visando um armazenamento consistente e acesso eficiente aos dados.
 - O processo inicia com a análise das relações não normalizadas que vão decompondo-se, através de uma série de etapas sucessivas, em novas relações. Estas novas relações obedecem uma série de normas.
 - As três principais normas são:
 - Primeira Forma Normal (1FN)
 - Segunda Forma Normal (2FN)
 - Terceira Forma Normal (3FN)

- **Projeto Lógico**

- **Primeira Forma Normal (1FN)** : A primeira forma normal indica que:

- Não pode haver tabelas aninhadas.
- As linhas não deve conter itens repetidos.
- Os atributos devem ser atômicos, ou seja, devem ser indivisíveis.
- Os atributos não contem valores nulos, ou seja, não pode haver colunas com valores nulos.

ATENÇÃO!! Atualmente, não há mais restrição quanto à existência de valores nulos em colunas de uma tabela.

- **Projeto Lógico**

- **Primeira Forma Normal (1FN)** : Como Obter a primeira forma Normal:

Idpaciente	Nome	Cpf	Endereço	Telefone	Data Consulta	hora	Medico	Especialidade	Cod
4	Antonio	456.445.8	Av. Serrana 1204 - Centro - Serra	876-9839	03/05/2024	15:24	Julio Cesar	Geriatra	2
					04/05/2024	15:24	Renato Mc	Cardiologista	3
1	carlos	111.222.3	Av. Eurico de aguar 256 - vila velha	333	03/05/2024	10:15	Jose Luis	pediatra	1
					03/05/2024	20:20	Jose Luis	pediatra	1
					10/05/2024	12:00	Renato Mc	Cardiologista	3
3	Jairo Lucas	333.321.3	Av. Gil Veloso 2290 apto 102 -Vitória	887-0987	10/05/2024	10:00	Renato Mc	Cardiologista	3
					10/05/2024	10:00	Renato Mc	Cardiologista	3
2	José Maria	222.123.4	Rua Joao da cruz 123/23 - Vitória	9876-3434	10/05/2024	10:00	Julio Cesar	Geriatra	2
5	Paulo Cesar	586.125.4	AV. Replendor 123 apt 1203 - Vila Velha	9998-9888	11/05/2024	10:00	Renato Mc	Cardiologista	3

- **Projeto Lógico**

- **Primeira Forma Normal (1FN) : Como Obter a primeira forma Normal:**

- Resolver o problema de tabelas aninhadas. Pode-se ter duas opções:
 - Repetir os dados para cada linha da tabela aninhada, criando uma redundância de dados
 - Construir uma nova tabela para cada tabela aninhada.
- Transformar os atributos compostos em atributos atômicos:
 - O campo endereço será decomposto em dois campos: Rua, bairro, cidade.
- Definir uma chave que permita identificar um registro.
 - **IDpaciente + data + hora**

Banco de Dados II: Revisão

- **Projeto Lógico**
 - Primeira Forma Normal (1FN) : OK

idpaciente	nome	cpf	Endereco	telefone	Data Consulta	hora	medico	especialidade	cod
4	Antonio	456.445.876-45	Av. Serrana 1204 - Centro - Serra	876-9839	2024-05-03	15:24	Julio Cesar	Geriatra	2
1	carlos	111.222.333-55	Av. Eurico de aguar 256 - vila velha	333	2024-05-03	10:15	Jose Luis	pediatra	1
1	carlos	111.222.333-55	Av. Eurico de aguar 256 - vila velha	333	2024-05-03	20:20	Jose Luis	pediatra	1
1	carlos	111.222.333-55	Av. Eurico de aguar 256 - vila velha	333	2024-05-10	12:00	Renato Moura	Cardiologista	3
3	Jairo Lucas	333.321.345-98	Av. Gil Veloso 2290 apto 102 -Vitória	887-0987	2024-05-10	10:00	Renato Moura	Cardiologista	3
2	José Maria	222.123.434-56	Rua Joao da cruz 123/23 - Vitória	9876-3434	2024-05-10	10:00	Julio Cesar	Geriatra	2
5	Paulo Cesar	586.125.443-34	AV. Replendor 123 apt 1203 - Vila Velha	9998-9888	2024-05-11	10:00	Renato Moura	Cardiologista	3

idpa.	nome	cpf	Rua	Bairro	Cidade	telefone	Data Consulta	hora	medico	especial...	cod
1	carlos	111.222.333-55	Av. Eurico de aguar 256	Itapo	Vila Velha	333	2024-05-03	10:15	Jose Luis	pediatra	1
1	carlos	111.222.333-55	Av. Eurico de aguar 256	Itapo	Vila Velha	333	2024-05-03	20:20	Jose Luis	pediatra	1
4	Antonio	456.445.876-45	Av. Serrana 1204	Centro	Serra	876-9839	2024-05-03	15:24	Julio Cesar	Geriatra	2
3	Jairo Lucas	333.321.345-98	Av. Gil Veloso 2290 apto 102	Praia	Vitoria	887-0987	2024-05-10	10:00	Renato Moura	Cardiol...	3
1	carlos	111.222.333-55	Av. Eurico de aguar 256	Itapo	Vila Velha	333	2024-05-10	12:00	Renato Moura	Cardiol...	3
2	José Maria	222.123.434-56	Rua Joao da cruz 123/23	Penha	Vitoria	9876-3434	2024-05-10	10:00	Julio Cesar	Geriatra	2
5	Paulo Cesar	586.125.443-34	AV. Replendor 123 apt 1203	Gil Veloso	Vila Velha	9998-9888	2024-05-11	10:00	Renato Moura	Cardiol...	3

- **Projeto Lógico**

- **Primeira Forma Normal (1FN) : Problemas que persistem:**

- Somente é possível cadastrar um paciente se incluirmos um consulta para ele.
- Ao se deletar uma consulta (por exemplo consulta 3), perde-se todas as informações do paciente.
- Para alterar os dados de um paciente ou medico, será necessário fazer a alteração em vários registros

- **Projeto Lógico**

- **Segunda Forma Normal (2FN)** : A segunda forma normal diz que:

- Para aplicar a 2FN a tabela deve estar na 1FN.
- Cada coluna não chave, deve ser dependente da chave primaria completa, ou seja, não pode haver **dependências funcionais** parciais entre as colunas da tabelas.
 - Dependência funcional ocorre quando uma determinada coluna C2 tem seu valor dependente de uma coluna C1 ou de um conjunto de colunas (chave), ou seja, sempre que houver um valor em C1 haverá um valor correspondente em C2.
 - Dependência parcial : É quando uma coluna depende de apenas uma parte da chave.

- **Projeto Lógico**
 - **Segunda Forma Normal (2FN) :**
 - Dependências funcionais:

Chave definida

A	B	C	D
B	5	2	20
C	4	2	15
B	6	7	20
B	5	2	20
C	2	2	15
C	4	2	15
A	10	5	18
A	12	3	18
A	10	5	18
B	5	2	20
C	4	2	15
A	10	5	18
C	4	2	15

A coluna C depende da chave primária completa (A+B):

* Quando a chave for B5 a coluna C = 2

A coluna D depende apenas de PARTE da chave primária (apenas da coluna A) – Dependência parcial:

* Quando o valor da coluna A for B, a coluna D=20.

- **Projeto Lógico**

- **Segunda Forma Normal (2FN):** Como Obter:

- Analisar cada coluna, e ver se a mesma é dependente da chave primária. Em caso positivo, ela depende da chave inteira ou parte da chave?
- Se a coluna depender somente de parte da chave:
 - Criar (se não existir) uma nova tabela que tenha como chave primária a parte da chave que define a coluna e inserir a coluna na tabela.
 - Excluir a coluna da tabela original

- **Projeto Lógico**
 - Segunda Forma Normal (2FN): OK.

data	horario	medico	cod_medico	especialidade	fkpaciente
2024-05-03	10:15	Jose Luis	1	pediatra	1
2024-05-03	20:20	Jose Luis	1	pediatra	1
2024-05-03	15:24	Jose Luis	1	pediatra	4
2024-05-10	10:00	Jose Luis	1	pediatra	3
2024-05-10	12:00	Jose Luis	1	pediatra	1
2024-05-10	10:00	Jose Luis	1	pediatra	2
2024-05-11	10:00	Jose Luis	1	pediatra	5
2024-05-03	10:15	Paulo	3	Pediatra	1
2024-05-03	20:20	Paulo	3	Pediatra	1
2024-05-03	15:24	Paulo	3	Pediatra	4

idpaciente	cpf	Endereco	Telefone	nome
1	111.222.333-55	Av. Eurico de aguar 256 - vila velha	333	carlos
2	222.123.434-56	Rua Joao da cruz 123/23 - Vitória	9876-3434	José Maria
3	333.321.345-98	Av. Gil Veloso 2290 apto 102 -Vitória	887-0987	Jairo Lucas
4	456.445.876-45	Av. Serrana 1204 - Centro - Serra	876-9839	Antonio
5	586.125.443-34	AV. Replendor 123 apt 1203 - Vila Velha	9998-9888	Paulo Cesar

- **Projeto Lógico**

- **Terceira Forma Normal (3FN):** A terceira forma normal diz que:

- A tabela deve estar na segunda forma normal (2fN)

- **Dependências transitivas:** Uma coluna não-chave **não pode** depender funcionalmente de outro atributo não-chave que por sua vez dependa da chave

- Uma situação em que um atributo não-chave (A) depende de outro atributo não-chave (B), que por sua vez depende diretamente da chave primária (C).

- **NÃO** deve haver **dependências funcionais** entre atributos não-chave, ou seja, um atributo **não chave** não pode depender de outro atributo **não chave** (mesmo que este atributo não dependa da chave primária)

- Uma situação em que um atributo não-chave (A) depende de outro atributo não-chave (B).

- **Projeto Lógico**

- **Terceira Forma Normal (3FN):** Obter a 3FN:

- Analisar todas as colunas que não sejam chaves primárias e ver se o seu valor pode ser determinado a partir de outra coluna que não seja a chave primária. (dependência funcional)
 - Em caso positivo, criar uma nova tabela (se não existir) e copiar para ela a coluna analisada e a coluna que determinou seu valor.
 - A chave primária da nova tabela será a chave que determinou o valor da coluna analisada
 - Excluir da tabela original a coluna analisada.
- Copiar para a nova tabela todas as colunas que possam ser determinadas transitivamente pela nova chave
- Excluir essas colunas da tabela original.

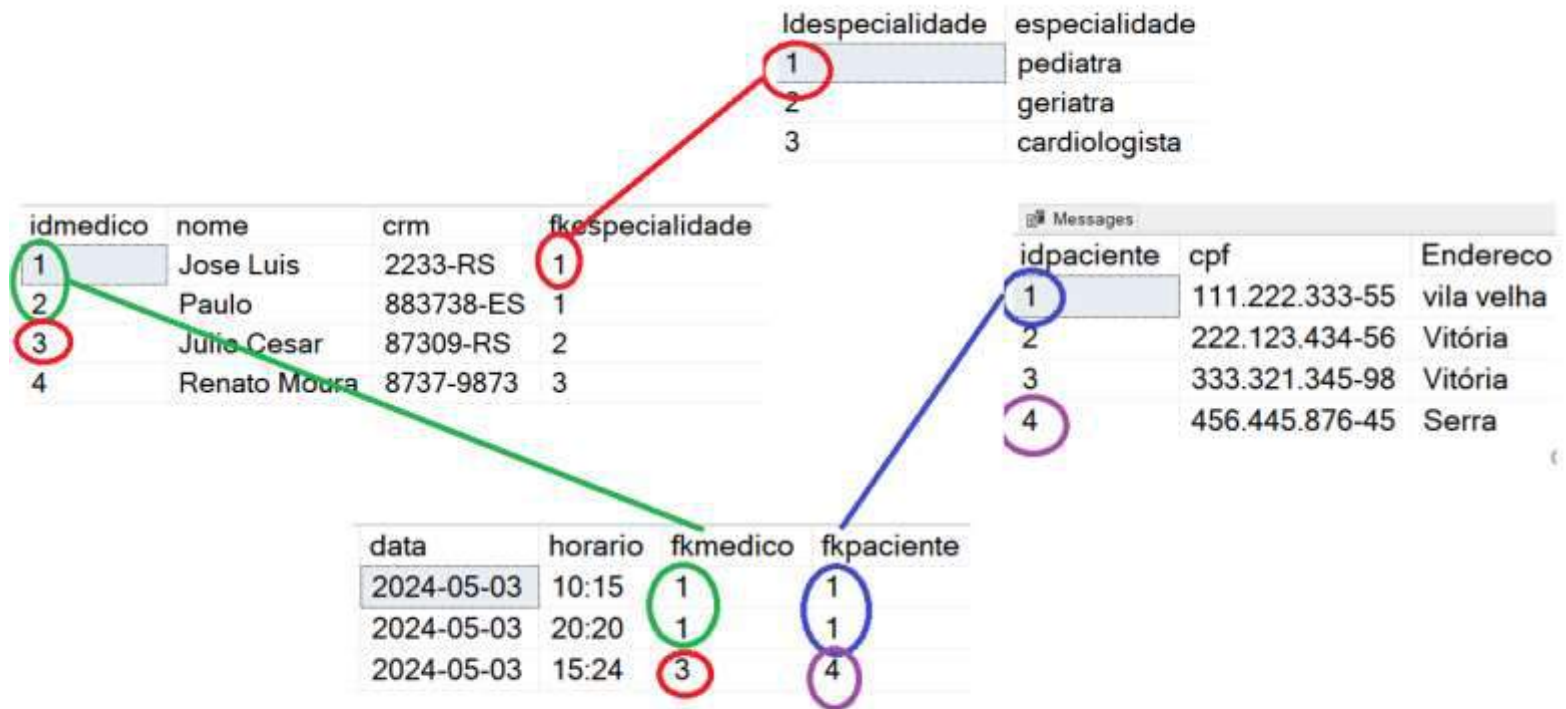
- **Projeto Lógico**
 - Terceira Forma Normal (3FN): **OK**

idmedico	crm	especialidade	nome	cod
1	2233-RS	pediatra	Jose Luis	1
2	883738-ES	Pediatra	Paulo	1
3	87309-RS	Geriatra	Julio Cesar	2
4	8737-9873	Cardiologista	Renato Moura	3

data	horario	fkmedico	fkpaciente
2024-05-03	10:15	1	1
2024-05-03	20:20	1	1
2024-05-03	15:24	3	4

Messages		
idpaciente	cpf	Endereco
1	111.222.333-55	vila velha
2	222.123.434-56	Vitória
3	333.321.345-98	Vitória
4	456.445.876-45	Serra

- **Projeto Lógico**
 - Terceira Forma Normal (3FN): **OK**



- **Projeto Lógico**
 - **Quarta Forma Normal (4FN):** quarta forma normal diz que:
 - Uma tabela deve estar na Terceira Forma Normal (3FN).
 - Não deve haver dependências multivaloradas não triviais .

- **Projeto Lógico**

- **Quarta Forma Normal (4FN):** quarta forma normal diz que:

CursosID	Professor	Livro
1	Ana	Álgebra
1	Ana	Cálculo
1	Bruno	Álgebra
1	Bruno	Geometria
2	Carlos	Física
2	Carlos	Química

- **Dependência Multivalorada:** Para um dado CursosID, temos múltiplos professores e múltiplos livros. Os professores e os livros são independentes entre si, mas ambos dependem de Curso

- **Projeto Lógico**

- Quarta Forma Normal (4FN): -

- Para alcançar a 4FN, você dividiria a tabela em duas outras tabelas:

CursoProfessores:

CursoID

Professor

CursoLivros:

CursoID

Livro

Banco de Dados II

- Linguagem na prática

SQL SERVER

Banco de Dados II: SQL SERVER

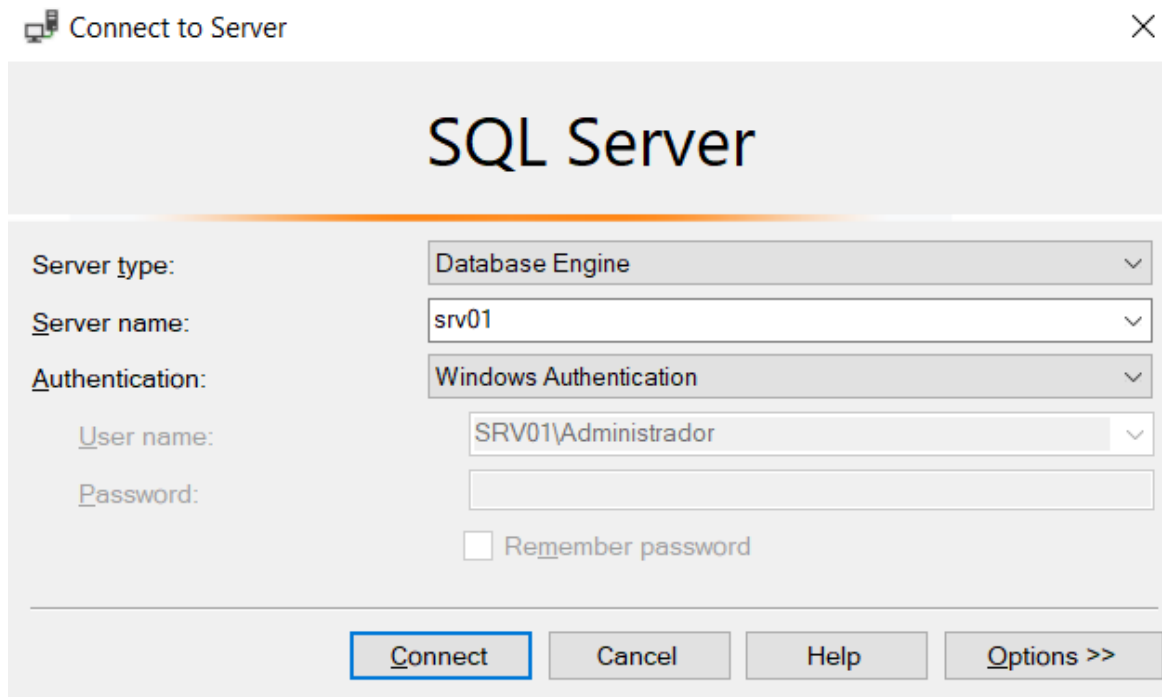
- O SQL Server é um SGBD cliente-Servidor fornecido pela Microsoft.
- Atualmente é o SGBD que detém a maior parcela de mercado.
- Possui várias versões diferentes:
 - **SQL Server 2022 Standard:** Esta edição fornece recursos básicos de banco de dados, incluindo armazenamento de dados, consultas, segurança e gerenciamento. É adequada para pequenas e médias empresas com necessidades de banco de dados mais simples.
 - **SQL Server 2022 Enterprise:** Esta edição oferece recursos avançados para grandes empresas e cargas de trabalho de missão crítica. Inclui recursos como compressão de dados, replicação, particionamento de tabelas, data warehousing e muito mais.
 - **SQL Server 2022 Developer:** Destinada para desenvolvedores de software que desejam criar e testar aplicativos com todos os recursos disponíveis nas edições Enterprise e Standard do SQL Server. Esta edição é idêntica à Enterprise em termos de recursos, mas licenciada para uso somente em ambientes de desenvolvimento e teste.
 - **SQL Server 2022 Express:** Uma versão gratuita e limitada do SQL Server, ideal para desenvolvimento de pequenos aplicativos, implantações leves de produção e aprendizado. Tem limitações de tamanho de banco de dados e recursos em comparação com as edições Standard e Enterprise.

Banco de Dados II: SQL SERVER

- Possui várias versões diferentes:
 - **SQL Server 2022 Web:** Esta edição é projetada para implantações de hospedagem na web e inclui recursos específicos para hospedar bancos de dados em ambientes de servidor web.
 - **SQL Server 2022 Business Intelligence:** Esta edição é voltada para soluções de inteligência de negócios, fornecendo ferramentas e recursos específicos para análise de dados, relatórios e integração de dados.

Banco de Dados II: SQL SERVER

- **SQL Server Management Studio (SSMS):** É ferramenta de administração e desenvolvimento desenvolvida pela Microsoft para gerenciar instâncias do SQL.
 - Ele fornece uma interface gráfica robusta e abrangente para administradores de banco de dados, desenvolvedores e analistas de dados realizarem uma variedade de tarefas.



Connect to Server

SQL Server

Server type: Database Engine

Server name: srv01

Authentication: Windows Authentication

User name: SRV01\Administrador

Password:

☐ Remember password

Connect Cancel Help Options >>

Banco de Dados II: SQL SERVER

- **SQL Server Management Studio (SSMS):**
- **Tipo de serviço :** indica o tipo de serviço que o usuário quer iniciar (**Data Engine – O gerenciador bancos**, Report Servide, Azure..)
- **Server Name :** Indica a qual servidor SQL usuário quer se conectar
- **Authentication :** Indica o tipo de autenticação usada para validar o usuário (windows ou Sql)
 - **Autenticação Windows:** As credenciais de autenticação são verificadas pelo sistema operacional Windows. Isso significa que o SQL Server delega a responsabilidade de autenticação ao ambiente Windows subjacente. O usuário precisa ter uma conta de usuário do Windows válida.
 - **Autenticação Sql:** As credenciais de autenticação são verificadas internamente pelo SQL Server. O usuário fornece um nome de usuário e senha diretamente ao SQL Server para autenticação.

SQLQuery1.sql - srv01.lucas (SRV01\Administrador (61)) - Microsoft SQL Server Management Studio (Administrator)

File Edit View Query Project Tools Window Help

1

lucas

Execute

sp_i_validaAcesso

Object Explorer

Connect

2

srv01 (SQL Server 15.0.2104 - SRV01\Administrador)

Databases

System Databases

Database Snapshots

Artsoft

BelloMatriz

hawaiiMatriz

Imagens

lucas

Nfe_uniao

SisfrutasArtsoft

sisfrutasbello

SisfrutasHawai

Sisfrutos

sisfrutos_faz

Sisfrutos_Uniao

UtArtsoft

Security

Server Objects

Replication

PolyBase

Management

XFEvent Profiler

SQLQuery1.sql - srv...Administrador (61))

3

select * from pagar

194 %

Results Messages

	IdPagar	Pag_Fatura	Pag_Parcela	Pag_Descricao	Pag_Confirmado	Pag_DiEmissao
1	2	Aut000001	02/12	aluguel	0	2006-01-01 00:00:00.000
2	3	Aut000001	03/12	aluguel	0	2006-01-01 00:00:00.000
3	4	Aut000001	04/12	aluguel	0	2006-01-01 00:00:00.000
4	5	Aut000001	05/12	aluguel	0	2006-01-01 00:00:00.000
5	6	Aut000001	06/12	aluguel	0	2006-01-01 00:00:00.000
6	7	Aut000001	07/12	aluguel	0	2006-01-01 00:00:00.000
7	8	Aut000001	08/12	aluguel	0	2006-01-01 00:00:00.000

4

Banco de Dados II: SQL SERVER

■ SQL Server Management Studio (SSMS):

- **1 – Base de Dados:** Mostra base de dados atualmente em uso. Também permite selecionar a base desejada
- **2- Objeto explorer:** Área onde é mostrado todos os bancos de dados attached no servidor e demais objetos do banco.
- **3 – Área das “queries”:** Área utilizada para enviar comandos para o SQL Server.
- **4– Área de mensagens e resultados:** Área que mostra o resultado do comando executado e as mensagens relativas ao mesmo

Banco de Dados II: SQL SERVER

- Criar base de dados:
 - Utilizando comandos SQL.

```
CREATE DATABASE Teste
    --- criar arquivo de dados
ON
( NAME = 'Teste_dat', --nome Logico do arquivo
  FILENAME = 'c:\lixo\Testedat.mdf', -- caminho e nome do arquivo fisico
  SIZE = 10MB, -- Tamnho inicial do arquivo, pode ser em MB, GB, TR
  MAXSIZE = 50MB, -- Tamanho maximo do arquivo (opcional)
  FILEGROWTH = 10% -- Como será o crescimento do arquivo. Pode ser em MB, %...
)
--Cria arquivo de LO
LOG ON
( NAME = Teste_log,
  FILENAME = 'c:\lixo\teste_log.ldf',
  SIZE = 5MB,
  MAXSIZE = 25MB,
  FILEGROWTH = 10%
)
COLLATE Latin1_General_CI_AS -- Define a linguagem default do banco. Deve ser dado
                             -- especial importancia ao CI e AS. Estas siglas definem
                             -- se o banco sera (CS / AS) ou não (CI/AI) diferenciar
                             -- letras maiúsculos e minisculo e os acentos
```


Banco de Dados II: SQL SERVER

■ Criar Tabelas:

```
CREATE TABLE [dbo].[Matricula](  
    [Idmatricula] [int] IDENTITY(1,1) NOT NULL,  
    [mat_frequencia] [int] NULL default 0,  
    [mat_nota] [numeric](18, 2) NULL check (mat_nota >=0),  
    [fkaluno] [int] NOT NULL,  
    [fkdisciplina] [int] NOT NULL,  
    CONSTRAINT [PK_Disciplina] PRIMARY KEY CLUSTERED  
    ( Idmatricula ASC),  
  
    CONSTRAINT FK_Matricula_Aluno FOREIGN KEY (fkaluno)  
        REFERENCES Aluno(idaluno),  
  
    CONSTRAINT FK_Matricula_Disciplina FOREIGN KEY (fkdisciplina)  
        REFERENCES Disciplina (Iddisciplina)
```


Banco de Dados II: SQL SERVER

■ Tipos de dados do SQL Server:

Tipos de Dados Numéricos:

Inteiros: TINYINT, SMALLINT, INT, BIGINT

Numéricos de Ponto Flutuante: FLOAT, REAL

Decimais e Numéricos: DECIMAL, NUMERIC

Tipos de Dados de Data e Hora:

DATE: Armazena data no formato AAAA-MM-DD.

TIME: Armazena hora do dia no formato hh:mm:ss.nnnnnnnn.

DATETIME: Armazena data e hora no formato AAAA-MM-DD hh:mm:ss.

TIMESTAMP: Sinônimo para ROWVERSION, utilizado para versionamento de linha.

Banco de Dados II: SQL SERVER

■ Tipos de dados do SQL Server:

Tipos de Dados de Caracteres:

Caracteres Fixos: CHAR(n), onde n é o número de caracteres.

Caracteres Variáveis: VARCHAR(n) ou VARCHAR(MAX) para textos longos.

Caracteres Fixos Unicode: NCHAR(n), suporta armazenamento de caracteres Unicode.

Caracteres Variáveis Unicode: NVARCHAR(n) ou NVARCHAR(MAX).

Tipos de Dados Binários:

Binários Fixos: BINARY(n)

Binários Variáveis: VARBINARY(n) ou VARBINARY(MAX)

Banco de Dados II: SQL SERVER

■ Tipos de dados do SQL Server:

Outros Tipos de Dados:

XML: Armazena dados XML.

JSON: Embora o SQL Server não tenha um tipo de dados dedicado para JSON, ele suporta armazenamento de JSON em colunas de tipo de dados de string (VARCHAR, NVARCHAR) e oferece funções para trabalhar com JSON.

UNIQUEIDENTIFIER: Armazena um identificador único global (GUID).

SQL_VARIANT: Pode armazenar dados de vários tipos, exceto TEXT, NTEXT, TIMESTAMP, e alguns outros.

Banco de Dados II: SQL SERVER

■ Criar Tabelas:

- Inserir chaves após criação da tabela

```
ALTER TABLE Matricula ADD CONSTRAINT FK_Matricula_Disciplina FOREIGN KEY(fkdi  
REFERENCES Disciplina (Iddisciplina)  
on Update NO ACTION  
on delete NO ACTION  
GO
```

Banco de Dados II: SQL SERVER

■ Excluir e manipular objetos:

```
Drop Table Aluno
```

```
Alter Table Aluno drop constraint FK_Aluno_curso
```

```
Alter Table Aluno drop column alu_nome
```

```
DROP INDEX IX_aluno_nome ON Aluno
```

Banco de Dados II: SQL SERVER

■ DML

● Comando INSERT

```
insert into <nome_tabela>
(campo1,
 campo2)
Values
(valor1,
 valor2')
```

```
insert into aluno
(alu_nome,
 alu_cpf)
Values
('jairo',
 '112.222.222-3')
```

Banco de Dados II: SQL SERVER

■ DML

● Comando INSERT

```
INSERT INTO aluno
  (alu_nome, alu_cpf)
VALUES
  ('Maria Silva', '112.222.222-3'),
  ('João Pereira', '112.222.222-3'),
  ('Ana Costa', '112.222.222-3');
```


Banco de Dados II: SQL SERVER

■ DML

● Comando INSERT

```
INSERT INTO aluno  
    (alu_nome,  
    alu_cpf)  
(select alu_nome,  
    alu_cpf  
from aluno)
```

Comando INSERT permite inserir dados a partir de uma consulta sql. Neste caso, não se a clausula **Value**

Banco de Dados II: SQL SERVER

■ DML

● Comando Update

```
update aluno set  
    alu_cpf='1223233'
```

- O comando update geralmente é utilizado com alguma condição (**where**), pois caso contrário, a alteração seria efetuado em todos os registros da tabela (como no exemplo acima)

```
update aluno set  
    alu_cpf='1223233'  
where idaluno = 3
```

Banco de Dados II: SQL SERVER

■ DML

● Comando Delete

```
delete pagar  
where idpagar =1
```

```
delete pagar  
from empresa  
where fkempresa = idempresa  
and Emp_RazaoSocial='Lucas'
```

- O comando delete geralmente é utilizado com alguma condição (**where**), pois caso contrário, a exclui TODOS os dados da tabela

Banco de Dados II: SQL SERVER

■ DML

● Comando Select

- É provavelmente o comando mais usado da linguagem Sql.
- O SELECT permite selecionar uma ou mais colunas de uma ou várias tabelas, visualizações, ou combinações destas, retornando os dados como um conjunto de resultados, geralmente em forma de tabela

Banco de Dados II: SQL SERVER

■ DML

● Comando Select

```
SELECT coluna1, coluna2, ..., colunaN  
FROM nome_da_tabela  
WHERE condicao_de_filtro  
GROUP BY colunas_de_agrupamento  
HAVING condicao_de_agrupamento  
ORDER BY colunas_de_ordenacao
```

Banco de Dados II: SQL SERVER

■ Para selecionar todos os campos

- `Select * from nome_tabela`

■ Usando restrição

- `Select * from colaborador
where col_nome='José'`

`Select * from colaborador
where idcolaborador<10`

Banco de Dados II: SQL SERVER

- Seleccionando campos de mais de uma tabela
 - `Select col_nome, col_cargo, dep_nome
from colaborador, dependente
where fkcolaborador=idcolaborador
and idcolaborador<10`
 - Importante: Quando se usa mais de uma tabela deve-se sempre lembrar de usar a clausula `where` para especificar qual será a ligação entre as tabelas.
 - Caso não for especificada esta ligação, será feita uma junção de cada registro da tabela 1 para registro da tabela 2.

Banco de Dados II : Revisão

	IdPai	Pai_nome	pai_cpf
1	1	José carlos	58601015033
2	2	Antonio	12301015033
3	3	Luiz fernando	12301015030

	Idfilho	Fil_nome	Fil_Datanascimento	Fil_sexo	fkpai
1	1	Luizinho	2005-10-12 00:00:00.000	M	2
2	2	Joaozinho	2009-10-12 00:00:00.000	M	2
3	4	Maria da penha	2004-10-12 00:00:00.000	F	1
4	5	Jose Maria	2007-10-12 00:00:00.000	M	1

	IdPai	Pai_nome	pai_cpf	Idfilho	Fil_nome	Fil_Datanascimento	Fil_sexo	fkpai
1	1	José carlos	58601015033	1	Luizinho	2005-10-12 00:00:00.000	M	2
2	1	José carlos	58601015033	2	Joaozinho	2009-10-12 00:00:00.000	M	2
3	1	José carlos	58601015033	4	Maria da penha	2004-10-12 00:00:00.000	F	1
4	1	José carlos	58601015033	5	Jose Maria	2007-10-12 00:00:00.000	M	1
5	2	Antonio	12301015033	1	Luizinho	2005-10-12 00:00:00.000	M	2
6	2	Antonio	12301015033	2	Joaozinho	2009-10-12 00:00:00.000	M	2
7	2	Antonio	12301015033	4	Maria da penha	2004-10-12 00:00:00.000	F	1
8	2	Antonio	12301015033	5	Jose Maria	2007-10-12 00:00:00.000	M	1
9	3	Luiz fernando	12301015030	1	Luizinho	2005-10-12 00:00:00.000	M	2
10	3	Luiz fernando	12301015030	2	Joaozinho	2009-10-12 00:00:00.000	M	2
11	3	Luiz fernando	12301015030	4	Maria da penha	2004-10-12 00:00:00.000	F	1
12	3	Luiz fernando	12301015030	5	Jose Maria	2007-10-12 00:00:00.000	M	1

`select * from pai,filho`

Banco de Dados II : Revisão

	IdPai	Pai_nome	pai_cpf
1	1	José carlos	58601015033
2	2	Antonio	12301015033
3	3	Luiz fernando	12301015030

	Idfilho	Fil_nome	Fil_Datanascimento	Fil_sexo	fkpai
1	1	Luizinho	2005-10-12 00:00:00.000	M	2
2	2	Joaozinho	2009-10-12 00:00:00.000	M	2
3	4	Maria da penha	2004-10-12 00:00:00.000	F	1
4	5	Jose Maria	2007-10-12 00:00:00.000	M	1

	IdPai	Pai_nome	pai_cpf	Idfilho	Fil_nome	Fil_Datanascimento	Fil_sexo	fkpai
1	2	Antonio	12301015033	1	Luizinho	2005-10-12 00:00:00.000	M	2
2	2	Antonio	12301015033	2	Joaozinho	2009-10-12 00:00:00.000	M	2
3	1	José carlos	58601015033	4	Maria da penha	2004-10-12 00:00:00.000	F	1
4	1	José carlos	58601015033	5	Jose Maria	2007-10-12 00:00:00.000	M	1

select * from pai,filho
WHERE fkpai=idpai

Banco de Dados II: SQL SERVER

■ Operador IN e NOT IN

- `Select * from pagar`

`where fkempresa NOT IN`

`(Select fkempresa from receber)`

- O comando acima mostraria todas as empresas que estivessem na tabela pagar e NÃO estivessem na tabela receber

- `Select * from pagar`

`where fkempresa IN (35,50,80,90)`

- Seleciona os registros do contas a pagar das empresas onde o campo fkempresa seja igual a a 35,50,80 e 90.

Banco de Dados II: SQL SERVER

■ Operador IS

- Para “comparar” qualquer valor com **nulo não se pode usar** o sinal de igualdade.

Select * from pagar where pag_dtpagamento=null

Select * from pagar where pag_dtpagamento<>null

- Ambas as consultas acima retornariam uma consulta vazia, ou seja, não retornariam nenhum registro.
- Para fazer consultas utilizando o valor null deve-se utilizar o operador IS.

Select * from pagar where pag_dtpagamento IS null

Select * from pagar where pag_dtpagamento IS NOT null

Banco de Dados II: SQL SERVER

■ Clausula AS

- Permite renomear o nome de uma coluna em determinado consulta
 - `Select emp_razaosocial AS nome_da_empresa, emp_cnpj AS CNPJ from empresa`

No resultado, será mostrado um campo nome_da_empresa e um campo CNPJ

- Extremamente útil para usar em funções de agregações como soma, média, qtd. Estas funções retornam um total em um coluna sem nome

Banco de Dados II: SQL SERVER

■ Funções de agregação:

- Sum (coluna) – Devolve a soma dos valores da coluna especificada
- MAX (coluna) – Devolve o MAIOR valor da coluna especificada
- MIN (coluna) – Devolve o MENOR valor da coluna especificada
- AVG (coluna) – Devolve o valor médio da coluna especificada
- Count(coluna) – Conta o número de registros NÃO NULOS da coluna especificada.

■ `Select count(pag_vlrpagto), sum(pag_vlrpagto), max(pag_vlrpagto), min(pag_vlrpagto), avg(pag_vlrpagto)`
from pagar

	(No column name)	(No column name)	(No column name)	(No column name)	(No column name)
1	5440	6360422.31	111966.95	0.00	1169.195277

■ `Select count(pag_vlrpagto) AS QtdReg, sum(pag_vlrpagto) AS Total, max(pag_vlrpagto) AS ValorMaximo, min(pag_vlrpagto) AS ValorMinimo, avg(pag_vlrpagto) AS ValorMedio`
from pagar

	QtdReg	Total	ValorMaximo	ValorMinimo	ValorMedio
1	5440	6360422.31	111966.95	0.00	1169.195277

Banco de Dados II: SQL SERVER

■ Funções de agregação:

- Funções de agregação retornam um único valor, portanto, não podem ser especificado os campos desejados sem especificar a clausula **group by**.

■ **select** NomeEmpresa, sum(pag_vlrpagto) **from** pagar

- Esta Consulta retornaria um erro, afinal, qual seria o nome da empresa que seria mostrado se estou somando o valor de TODOS os registros??
- Para este tipo de consulta, deve-se usar a clausula **GROUP BY**.

Banco de Dados II: SQL SERVER

■ Clausula GROUP BY

- O **GROUP BY** é utilizado sempre que se quer totalizar valores por grupos.

- Exemplo: A empresa deseja um relatório com o nome da empresa e o total de faturas ainda não pagas da mesma
 - `select emp_razaosocial, sum(pag_valor) as Valor from pagar, empresa where pag_dtpagto is null group by emp_razaosocial`
- O comando acima irá agrupar (somando o pag_valor) os registros onde o conteúdo do campo Emp_razaosocial sejam iguais.
- Filiais que possuem o mesmo nome seriam agrupadas em um único registro. Caso quiséssemos separar as filiais, deveríamos incluir o campo CNPJ no order by.

■ Clausula GROUP BY

- O **HAVING** é uma clausula de restrição (filtro) utilizado **DEPOIS** da execução do group by, ou seja, restrição é aplicada em cima do **RESULTADO** do comando do **group by**

select

fkempresa,

Emp_RazaoSocial,

sum(pag_valor) as total

from pagar,

empresa

where fkempresa=idempresa

group by fkempresa,

Emp_RazaoSocial

having **sum**(pag_valor) <1000

Banco de Dados II: SQL SERVER

■ Outros operadores de comparação

- $A > B$, $A \geq B$ - A maior B , A maior ou igual a B
- $A < B$, $A \leq B$ - A menor B A menor ou igual a B
- $A \text{ in } (B)$ - A esta contido em B
- $A \text{ Between } B$ - Intervalo entre A e B.
 - `Select * from pagar`
`where Data Between 01/02/2012 and 28/02/2012`
- Condição1 **AND** Condição2 - 'E' lógico. Ambas as condições devem ser verdadeiras
- Condição1 **OR** Condição2 - 'OU' lógico. Basta UMA condição ser verdadeira.

● $A \text{ like } \% B$ – A contém B

- `Select * from filho where filho_nome like 'Maria%'`
 - Mostraria os nomes que comesçassem com Maria. **Não** mostraria o nome 'José Maria', por exemplo
- `Select * from filho where filho_nome like '%Maria'`
 - Mostraria os nomes que TERMINASSEM com Maria. **Não** mostraria o nome 'Maria da penha', por exemplo
- `Select * from filho where filho_nome like '%Maria%'`
 - Mostraria os nomes que tivessem Maria. Mostraria tanto o nome 'Maria da penha', como o nome 'José Maria'

Banco de Dados II: Comandos e Funções SQL

■ Clausula Union / Union All

- Como o próprio nome sugere, Combina (une) o resultado de dois ou mais select's em uma única saída.
- Os select's que serão combinados devem ter o mesmo número de colunas, e os mesmos tipos de dados.
- O nome das colunas resultantes será o nome das colunas do primeiro select.
- Por default, o UNION elimina as linhas repetidas, caso queira manter estas linhas deve-se usar UNION ALL

Banco de Dados II: Comandos e Funções SQL

■ Clausula Union / Union All

- Select pag_data, pag_valor from pagar
union all
select rec_data, rec_valor from receber

O resultado será a união de todos os registros do tabela pagar e da tabela receber, SOMENTE com os campos selecionados.

Banco de Dados II: Comandos e Funções SQL

■ Clausula INTO

- Uma das formas mais usadas para criar e/ou copiar tabelas (temporárias ou não) baseadas em outra(s) tabela(s) ou em um select é a cláusula INTO.
- Exemplo:
`Select * INTO novatabela from pagar`

Cria uma tabela com o nome de novatabela contendo todos os campos e TODOS os dados da tabela Pagar.

CUIDADO: A clausula INTO não copia chaves primárias, índices ,relacionamentos, restrições...

E para copiar somente a estrutura de uma tabela, ou seja, não copiar os dados???

Banco de Dados II: Comandos e Funções SQL

■ Clausula JOIN

- A cláusula JOIN é usada para combinar linhas de duas ou mais tabelas, com base em uma condição relacionada, geralmente um campo comum entre as tabelas. Existem vários tipos de JOIN, incluindo INNER JOIN, LEFT JOIN, RIGHT JOIN, e FULL JOIN.
- **INNER JOIN:** Ou somente JOIN, é usado para combinar registros de duas ou mais tabelas em um banco de dados, baseando-se em uma coluna comum entre elas. O resultado é um novo conjunto de registros que inclui colunas de ambas as tabelas.

```
select idempresa, emp_razaosocial, pag_valor  
from pagar join empresa on fkempresa=idempresa
```

Banco de Dados II: Comandos e Funções SQL

■ Clausula JOIN

- **Left JOIN:** É usado para combinar registros de duas ou mais tabelas em um banco de dados, porém, vai mostrar **TODOS** os registros da tabela da **ESQUERDA** e os registros correspondentes da tabela da direita.
 - Quando não existe correspondência na tabela da direita, **os campos desta tabela são mostrados com o valor NULL**

```
select idempresa, emp_razaosocial, pag_valor  
from pagar LEFT join empresa on fkempresa=idempresa
```

- Se não houver correspondência na tabela da direita, a linha resultante incluirá todas as colunas da tabela da **ESQUERDA** e **NULL para as colunas da tabela da DIREITA.**

Banco de Dados II: Comandos e Funções SQL

■ Clausula JOIN

- **RIGHT JOIN:** É usado para combinar registros de **duas** ou mais tabelas em um banco de dados. Mostra **TODOS** os registros da tabela da **DIREITA** e os registros correspondentes da tabela da esquerda.
 - Quando não existe correspondência na tabela da esquerda, **os campos desta tabela são mostrados com o valor NULL**

```
select idempresa, emp_razaosocial, pag_valor  
from pagar RIGHT join empresa on fkempresa=idempresa
```

- Se não houver correspondência na tabela da esquerda, a linha resultante incluirá todas as colunas da tabela da **DIREITA** e **NULL para as colunas da tabela da ESQUERDA**.

Banco de Dados II: Comandos e Funções SQL

■ Conversão de Tipo de dados

- O Sql oferece duas funções de conversão:

- Cast :

- **Cast**(expressão/campo as tipodesejado)

- Convert :

- **Convert** (tipodesejado, expressão/campo,estilo)

Ambas tem o mesmo propósito, a diferença é que a função **Convert** permite especificar um estilo (formatação) de saída. É bastante usado em conversões de campos do tipo data.

Banco de Dados II: Comandos e Funções SQL

```
select pag_fatura, pag_descricao,  
       pag_dtvencimento, 'cliente pagou em '+cast(pag_dtpagto as varchar(12)) as status  
from pagar  
where pag_dtpagto is not null
```

	pag_fatura	pag_descricao	pag_dtvencimento	status
1	Aut000001	COMPRA DE GRAO	2009-10-28 00:00:00.000	cliente pagou em Out 28 2009
2	Aut000001	COMPRA DE GRAO	2009-11-04 00:00:00.000	cliente pagou em Nov 16 2009
3	Aut000001	COMPRA DE GRAO	2009-11-11 00:00:00.000	cliente pagou em Nov 16 2009
4	Man000009	COMBUSTIVEL	2009-10-16 00:00:00.000	cliente pagou em Out 19 2009
5	Man000010	COMPRA DE GRAO	2009-10-30 00:00:00.000	cliente pagou em Out 30 2009
6	Aut000011	UNIFORMES	2009-11-06 00:00:00.000	cliente pagou em Out 28 2009
7	Aut000011	UNIFORMES	2009-11-27 00:00:00.000	cliente pagou em Nov 30 2009

```
select pag_fatura, pag_descricao,  
       pag_dtvencimento, 'cliente pagou em '+convert(varchar(12),pag_dtpagto,104) as status  
from pagar  
where pag_dtpagto is not null
```

	pag_fatura	pag_descricao	pag_dtvencimento	status
1	Aut000001	COMPRA DE GRAO	2009-10-28 00:00:00.000	cliente pagou em 28.10.2009
2	Aut000001	COMPRA DE GRAO	2009-11-04 00:00:00.000	cliente pagou em 16.11.2009
3	Aut000001	COMPRA DE GRAO	2009-11-11 00:00:00.000	cliente pagou em 16.11.2009
4	Man000009	COMBUSTIVEL	2009-10-16 00:00:00.000	cliente pagou em 19.10.2009
5	Man000010	COMPRA DE GRAO	2009-10-30 00:00:00.000	cliente pagou em 30.10.2009
6	Aut000011	UNIFORMES	2009-11-06 00:00:00.000	cliente pagou em 28.10.2009

Banco de Dados II: Comandos e Funções SQL

1	101	EUA	1 = mm/dd/yy 101 = mm/dd/yyyy
2	102	ANSI	2 = yy.mm.dd 102 = yyyy.mm.dd
3	103	Britânico/francês	3 = dd/mm/yy 103 = dd/mm/yyyy
4	104	Alemão	4 = dd.mm.yy 104 = dd.mm/yyyy
5	105	Italiano	5 = dd-mm-yy 105 = dd-mm-yyyy
6	106 ¹	-	6 = dd mon yy 106 = dd mon yyyy
7	107 ¹	-	7 = Mon dd, yy 107 = Mon dd, yyyy
8 ou 24	108	-	hh:mm:ss

Banco de Dados II: Comandos e Funções SQL

■ Manipulação de dados

- O SQL oferece várias funções que permitem manipular dados, algumas são:
 - Getdate() = Retorna a hora atual
 - Select Getdate()
 - DateDiff (tipo intervalo, data inicial, data final) – Mostra o intervalo de tempo entre a data inicial e a data final. O intervalo de tempo pode ser:
 - yy – em anos
 - mm – em meses
 - dd – em dias
 - ww – em semanas
 - hh – em horas
 - mi – em minutos
 - ss – em segundos

Banco de Dados II: Comandos e Funções SQL

■ Manipulação datas

- DatePart (parte desejada, data) – Mostra a parte desejada de um data especificada. Utiliza a mesma nomenclatura do datediff para especificar a parte da data desejada.

```
Select datepart(yy, getdate()) as ano,  
       datepart(mm, getdate()) as mes,  
       datepart(dd, getdate()) as diaMes,  
       datepart(dw, getdate()) as diaSemana,  
       datepart(hh, getdate()) as hora,  
       datepart(mi, getdate()) as minuto
```

	ano	mes	diaMes	diaSemana	hora	minuto
1	2012	2	24	6	14	51

Banco de Dados II: Comandos e Funções SQL

■ Manipulação datas

- Datename (parte desejada, data) – Mesma função do datepart, porém mostra o nome do dia ou mês

```
Select datepart (mm, getdate ()) as mes,  
       datename (mm, getdate ()) as 'mes por extenso',  
       datepart (dw, getdate ()) as diaSemana,  
       datename (dw, getdate ()) as 'dia por extenso'
```

mes	mes por extenso	diaSemana	dia por extenso
2	Fevereiro	6	Sexta-Feira

Banco de Dados II: Comandos e Funções SQL

■ Manipulação dados

● Outras funções:

- Day(date) – dia mês da data especificada
- Month(data) – mês da data especificada
- Year(data) – Ano da data especificada

Dados Relacionais II: Comandos e funções

■ Tabelas temporárias

- O sql permite a criação de tabelas temporárias que são automaticamente excluídas quando a seção é encerrada.
- Estas tabelas ficam armazenadas em um banco específico, gerenciado pelo próprio sql, o banco TEMPDB.
- O procedimento para criação de uma tabela é exatamente o mesmo de uma tabela normal, a única diferença é que se deve acrescentar o sinal # antes do nome da tabela.

```
Create table #tabelatemporaria
(
    codigo          int null,
    nome            varchar(30) null,
    endereço        varchar(80) null
)
```

Dados Relacionais II: Comandos e funções

- Tabelas temporárias podem ter escopo local ou global
 - # : Acessíveis apenas na sessão em que foram criadas e são automaticamente descartadas quando a sessão é encerrada ou a conexão é fechada.
 - ## : Acessíveis de qualquer sessão ou conexão ao servidor, enquanto a sessão que as criou ainda está ativa ou enquanto elas ainda estão sendo referenciadas.
 - Ambas ficam armazenadas no banco de dados **tempdb**.

Dados Relacionais II: Comandos e funções

- Manutenção BD
 - Backup / Restore
 - DBCC

Dados Relacionais II: Comandos e funções

■ Manutenção BD: Backup's

■ Backup Completo (Full Backup)

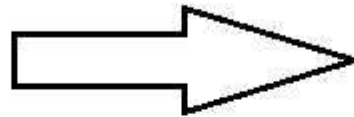
- **Descrição:** Efetua uma cópia completa de todo o banco de dados e parte do log de transações, de modo que o banco de dados possa ser restaurado para o estado em que estava no momento do backup.
- **Vantagens:** Cópia completa de todo o banco, fácil de implementar e restaurar pois todos os dados estão em um único arquivo.
- **Desvantagens:** O tempo para ser realizado pode ser relevante em bases de dados grandes, consome grande quantidade de espaço de armazenamento

Dados Relacionais II: Comandos e funções

Banco de dados
as 10:00d



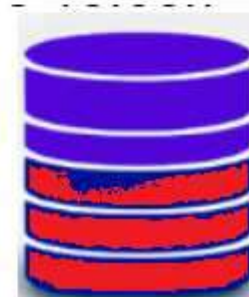
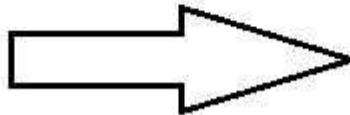
Backup
Completo



Banco de dados
as 18:00h



Backup
Completo



Dados Relacionais II: Comandos e funções

- Manutenção BD: Backup's

- **Backup Diferencial**

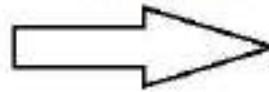
- **Descrição:** Captura todas as alterações feitas desde o **último backup completo**.
 - Cada backup diferencial sempre contém **todas as alterações** desde o último backup completo.
- **Vantagens:** mais rápido que o completo, visto que copia somente o que foi alterado, ocupa menos espaço de armazenamento, é razoavelmente fácil de restaurar, necessitando do backup completo e somente da última copia do backup incremental
- **Desvantagens:** Depende do backup completo para um restauração.

Dados Relacionais II: Comandos e funções

Banco de dados
as 10:00d



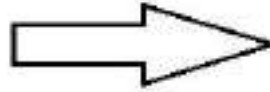
Backup
Completo



Banco de dados
as 18:00h



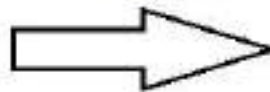
Backup
DIFERENCIAL



Banco de dados
as 22:00h



Backup
DIFERENCIAL



Dados Relacionais II: Comandos e funções

■ Manutenção BD: Backup's

■ Backup Incremental (**Não tem no Sql Server**)

- **Descrição** : Captura as alterações desde o **último backup feito** (seja completo, diferencial ou incremental).
- **Vantagens**: mais rápido e ocupa menos espaço que o completo e o diferencial. Poder ser realizado com mais frequência
- **Desvantagens**: Depende de toda uma cadeia de backup para a restauração. Se a cadeia “quebrar”, todo o restante do **backup é perdido**.
 - Completo às 08:00h (1)
 - Incremental as 9:00h (2)
 - Incremental as 10:00h (..)
 - Incremental as 20:00h (10)

(1) --> (2) --> (3) -->(4) -->.. (10)

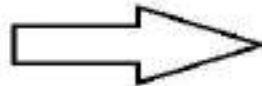
(1) --> (2) --> ~~(3)~~ -->~~(4)~~ -->..~~(10)~~

Dados Relacionais II: Comandos e funções

Banco de dados
as 10:00d



Backup
Completo



Banco de dados
as 18:00h



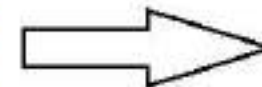
Backup
DIFERENCIAL



Banco de dados
as 23:00h



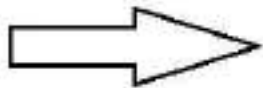
Backup
INCREMENTAL



Banco de dados
as 22:00h



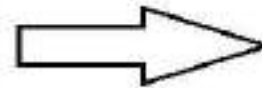
Backup
INCREMENTAL



Banco de dados
as 24:00h



Backup
INCREMENTAL



Dados Relacionais II: Comandos e funções

■ Manutenção BD: Backup's

■ Backup Logs

- **Descrição** : Captura todas as transações que ocorreram desde o último **backup de log** ou do último backup **COMPLETO**
- **Vantagens**: podem ser realizados muito frequentemente com impacto mínimo no desempenho. Restaura transação a transação.
- **Desvantagens**: Depende de toda uma cadeia de backup para a restauração. Se a cadeia “quebrar”, todo o **backup é perdido**.
 - Completo às 08:00h (1)
 - Backup log as 9:00h (2)
 - Backup log as 10:00h (3)
 - Backup log as 18:00h (4)

Restaurar:

(1) --> (2) --> (3) -->(4) -->.. (10)

(1) --> (2) --> ~~(3)~~ -->(4) -->.. (10)

Dados Relacionais II: Comandos e funções

Manutenção BD: Backup's – **Implementação**

- backup database <nomeBase> to disk= <caminho>
- backup database <nomeBase> to disk= <caminho>
with DIFFERENTIAL
- backup LOG <nomeBase> to disk=
<caminho>

Dados Relacionais II: Comandos e funções

Manutenção BD: Backup's – **Implementação**

■ Para ver as informações de um arquivo de backup

- RESTORE FILELISTONLY FROM DISK = <caminho do arquivo backup>

■ Restaurar backup

- RESTORE DATABASE <nome da base> FROM DISK = <caminho do arquivo de backup>

■ Restaurar backup com um nome / caminho diferente

- RESTORE DATABASE <nome da base> FROM DISK = 'caminho arquivo backup> WITH MOVE <nome_atual> TO <novo nome>, MOVE <Nome atual log> TO <novo nome LOG> RECOVERY

Dados Relacionais II: Comandos e funções

Manutenção BD: Backup's – **Implementação**

- backup database teste to disk= 'f:\lixo\bkp_teste_b.bkp'
with ENCRYPTION
(ALGORITHM = AES_256,
SERVER CERTIFICATE = Certificadoservidor)

Dados Relacionais II: Comandos e funções

Manutenção BD: – **Importação x Exportação**

■ **Importar dados via txt**

```
bulk insert <nome table>  
  from <caminho arquivo txt>  
WITH  
(  
  FIELDTERMINATOR =',' ,  
  ROWTERMINATOR ='\n'  
)
```

Dados Relacionais II: Comandos e funções

■ Manutenção BD

- **DBCC:** Os comandos DBCC (Database Console Commands) no SQL Server são ferramentas poderosas para administração de banco de dados, manutenção, e monitoramento.

■ Verificação de Integridade

- **DBCC CHECKDB:** Verifica a integridade física e lógica de todas as tabelas e índices em um banco de dados. É o mais abrangente dos comandos de verificação.
 - **DBCC CHECKDB (nomeBanco) with NO_INFOMSGS**
- **DBCC CHECKALLOC:** Verifica a consistência das estruturas de alocação de dados no banco de dados.
- **DBCC CHECKTABLE:** Verifica a integridade de todas as páginas de dados e índices de uma tabela específica ou de todas as tabelas em uma exibição indexada.

Dados Relacionais II: Comandos e funções

■ Manutenção BD

- **DBCC:** Os comandos DBCC (Database Console Commands) no SQL Server são ferramentas poderosas para administração de banco de dados, manutenção, e monitoramento.

■ Manutenção de Índices

- **DBCC DBREINDEX:** Reconstrói um ou mais índices de uma tabela.
 - DBCC DBREINDEX (nomeTabela) (obsoleto)
 - ALTER INDEX ALL ON <nomeTabela> REORGANIZE;
 - ALTER INDEX ALL ON <nomeTabela> Rebuild;
- **DBCC SHOWCONTIG (tabela):** Exibe informações sobre a fragmentação de índices na tabela especificada.

Dados Relacionais II: Comandos e funções

■ Manutenção BD

- **DBCC:** Os comandos DBCC (Database Console Commands) no SQL Server são ferramentas poderosas para administração de banco de dados, manutenção, e monitoramento.

■ Manutenção de espaço em disco

- **DBCC SHRINKDATABASE:** Reduz o tamanho de arquivos de dados e de log em um banco de dados específico
 - **DBCC SHRINKDATABASE('teste')**
- **DBCC SHRINKFILE:** Reduz o tamanho de um arquivo de dados ou de log específico.

Dados Relacionais II: Comandos e funções

■ Manutenção BD

- **DBCC:** Os comandos DBCC (Database Console Commands) no SQL Server são ferramentas poderosas para administração de banco de dados, manutenção, e monitoramento.

■ Informações do servidor sql

- **DBCC SQLPERF:** para monitorar o desempenho do servidor e reiniciar contadores
 - **DBCC SQLPERF(LOGSPACE)**
 - **DBCC SQLPERF('sys.dm_io_virtual_file_stats', CLEAR);**

Dados Relacionais II: Comandos e funções

■ Manutenção BD

- **DBCC:** Os comandos DBCC (Database Console Commands) no SQL Server são ferramentas poderosas para administração de banco de dados, manutenção, e monitoramento.

■ Manutenção tabelas

- **DBCC CHECKIDENT:** usado para verificar e, se necessário, corrigir problemas com o contador de identidade de uma tabela que possui uma coluna de identidade.

- **DBCC CHECKIDENT** ('nome_da_tabela', RESEED , 'novo valor')

DBCC HELP('?') / DBCC HELP('CHECKDB')

Dados Relacionais II: Comandos e funções

- Manutenção BD

- Outros comandos para monitoramento:

- ```
select database_id,
 name,
 cast((size * 8.0) / 1024 as numeric(12,1)) as Mb
from sys.master_files
```

- ```
select SO.name as Tabela, SI.rows  
from sys.sysobjects SO,  
     sys.sysindexes SI  
where SO.xtype='U'  
      and SI.ID=SO.ID  
      and indid<2
```

Dados Relacionais II: Comandos e funções

■ Manutenção BD

- Outros comandos para monitoramento:

SELECT

sqlserver_start_time AS 'Hora de Início do Servidor',

GETDATE() AS 'Hora Atual',

DATEDIFF(hh, sqlserver_start_time, GETDATE()) AS 'horas
Ativos'

FROM sys.dm_os_sys_info

Dados Relacionais II: Comandos e funções

■ Manutenção BD

- Outros comandos para monitoramento:

-- tempo que o usuário está conectado

SELECT

login_name AS Usuario,

AVG(DATEDIFF(HOUR, login_time, GETDATE())) AS
TempoMedioConexaoMinutos

FROM sys.dm_exec_sessions

GROUP BY login_name

Dados Relacionais II: Comandos e funções

■ Manutenção BD

- Outros comandos para monitoramento:

-- Transações pendentes

```
SELECT *
```

```
FROM sys.dm_tran_active_transactions
```

```
WHERE transaction_type = 1
```

Exemplo: transações ativas por mais de 5 minutos

```
SELECT
```

```
    transaction_id, database_id,
```

```
    DATEDIFF(MINUTE, transaction_begin_time, GETDATE()) AS
```

```
    DurationInMinutes, transaction_type, transaction_state, transaction_status
```

```
FROM sys.dm_tran_database_transactions WHERE transaction_state = 1
```

```
AND DATEDIFF(MINUTE, transaction_begin_time, GETDATE()) > 5
```

Banco de Dados II : Programação

- Programação SQL

Banco de Dados II : Programação

■ Comando IF

- O IF é um comando de desvio condicional. Tem a mesma sintaxe da maioria das linguagens:

- IF <condição>
 Begin
 comando...
 comando...
 end

OU

- IF <condição>
 Begin
 comando...
 comando...
 end
ELSE
 Begin
 comando...
 comando...
 end

Dados Relacionais (II): Programação

■ Case

- A expressão CASE permite avaliar um conjunto de expressão booleanas e retornar um determinado valor.

■ Case *expressão avaliada*

when valor1 *then* resultado1

when valor2 *then* resultado2

when valor3 *then* resultado3

End

Ou

■ Case

expressão booleana *when* valor1 *then* resultado1

expressão booleana *when* valor2 *then* resultado2

expressão booleana *when* valor3 *then* resultado2

end

Dados Relacionais (II): Programação

■ Case

Select aluno, turma,

status= Case

when nota < 4 then 'reprovado'

when nota > 4= and nota < 6 then 'Exame'

when nota >= 6 then aprovado

end

From alunos

- O Case pode ser usado com outros comandos como o Update

Banco de Dados II : Programação

■ Comando DECLARE e SET

- É possível utilizar variáveis no SQL. A sintaxe é a seguinte:
 - Declare @nome_variavel tipo_variavel
 - Declare @vnome varchar(50)
 - Declare @vvalor int
 - Declare @vvalor numeric(18,2)
- Para atribuir um valor a uma variável, é utilizado o comando SET
 - Set @vnome = 'Lucas'
 - Set @vvalor = 123
- Também é possível atribuir um valor a uma variável utilizando o comando select
 - Select @vnome = 'Lucas'
 - Select @vvalor = 123
 - Select @vvalor=pag_valor from pagar

Banco de Dados II : Programação

■ Comando DECLARE e SET

- As variáveis criadas podem ser utilizadas como parâmetros para comandos, como retorno ou mesmo em cálculos com outros campos.

```
Declare @totalgeral numeric(18,2)

select @totalgeral = sum(pag_valor)
  from pagar
 where pag_dtpagto is null

select emp_razaosocial,
       sum(pag_valor) as totalempresa ,
       (sum(pag_valor)/@totalgeral)*100 as '% do valor em atraso'
  from pagar,empresa
   where fkempresa=idempresa
   and pag_dtpagto is null
1  group by emp_razaoSocial
   order by totalempresa DESC
```

Banco de Dados II : Programação

■ Comando DECLARE e SET

Results		Messages	
	emp_razaosocial	totalempresa	% do valor em atraso
1	BANCO DO BRASIL	717363.73	26.750900
2	CAIXA ECONOMICA FEDERAL	397400.61	14.819300
3	POLO IND COM LTDA	298887.27	11.145600
4	RIO POLIMEROS S/A	212778.25	7.934600
5	FILM TRADING IMPORTAÇÃO E REPRESENTAÇÃO LTDA	152169.96	5.674500
6	DARF	142436.94	5.311500
7	BANCO BRADESCO S/A	126129.60	4.703400
8	TOTAL COM IMP E EXP DE TERMOPLASTICOS LTDA	115746.16	4.316200
9	ESCELSA	57642.32	2.149500
10	TIV PLASTICOS LTDA	41860.90	1.561000
11	BANCO VOLKSWAGEN SA	39873.68	1.486900
12	GPS	32427.44	1.209200

- E se eu quiser ver somente as empresas que representam mais de 5% do valor total em atraso???

Banco de Dados II: Comandos e funções

- Manutenção BD
 - Backup / Restore
 - DBCC

Banco de Dados II: Comandos e funções

■ Manutenção BD: Backup's

■ Backup Completo (Full Backup)

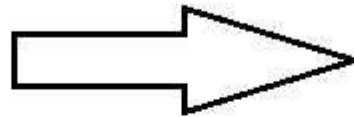
- **Descrição:** Efetua uma cópia completa de todo o banco de dados e parte do log de transações, de modo que o banco de dados possa ser restaurado para o estado em que estava no momento do backup.
- **Vantagens:** Cópia completa de todo o banco, fácil de implementar e restaurar pois todos os dados estão em um único arquivo.
- **Desvantagens:** O tempo para ser realizado pode ser relevante em bases de dados grandes, consome grande quantidade de espaço de armazenamento

Banco de Dados II: Comandos e funções

Banco de dados
as 10:00d



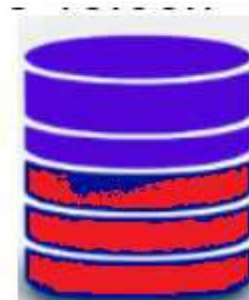
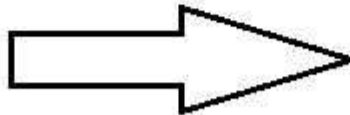
Backup
Completo



Banco de dados
as 18:00h



Backup
Completo



Banco de Dados II: Comandos e funções

- Manutenção BD: Backup's

- **Backup Diferencial**

- **Descrição:** Captura todas as alterações feitas desde o **último backup completo**.

- Cada backup diferencial sempre contém **todas as alterações** desde o último backup completo.

- **Vantagens:** mais rápido que o completo, visto que copia somente o que foi alterado, ocupa menos espaço de armazenamento, é razoavelmente fácil de restaurar, necessitando do backup completo e somente da última copia do backup incremental

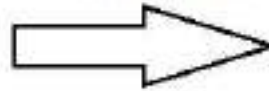
- **Desvantagens:** Depende do backup completo para um restauração.

Banco de Dados II: Comandos e funções

Banco de dados
as 10:00d



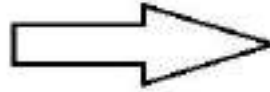
Backup
Completo



Banco de dados
as 18:00h



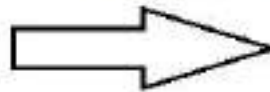
Backup
DIFERENCIAL



Banco de dados
as 22:00h



Backup
DIFERENCIAL



Banco de Dados II: Comandos e funções

■ Manutenção BD: Backup's

■ Backup Incremental (**Não tem no Sql Server**)

- **Descrição** : Captura as alterações desde o **último backup feito** (seja completo, diferencial ou incremental).
- **Vantagens**: mais rápido e ocupa menos espaço que o completo e o diferencial. Poder ser realizado com mais frequência
- **Desvantagens**: Depende de toda uma cadeia de backup para a restauração. Se a cadeia “quebrar”, todo o restante do **backup é perdido**.
 - Completo às 08:00h (1)
 - Incremental as 9:00h (2)
 - Incremental as 10:00h (..)
 - Incremental as 20:00h (10)

(1) --> (2) --> (3) -->(4) -->.. (10)

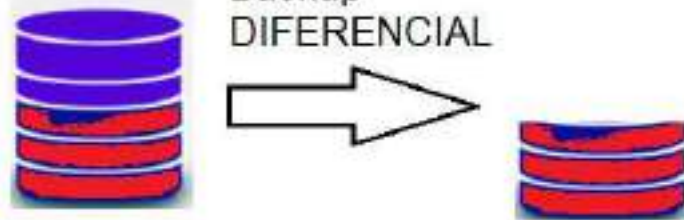
(1) --> (2) --> ~~(3)~~ -->~~(4)~~ -->..~~(10)~~

Banco de Dados II: Comandos e funções

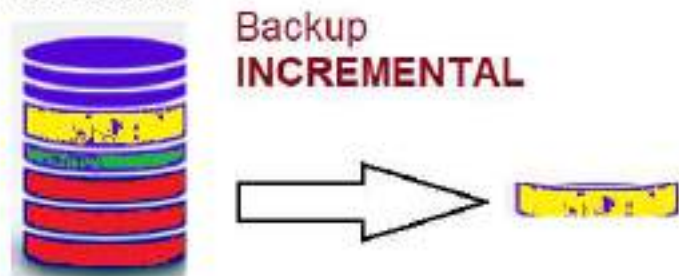
Banco de dados
as 10:00d



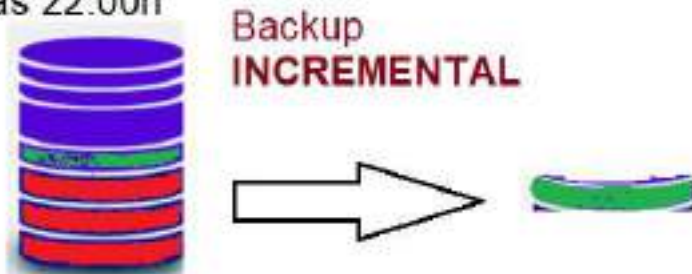
Banco de dados
as 18:00h



Banco de dados
as 23:00h



Banco de dados
as 22:00h



Banco de dados
as 24:00h



Banco de Dados II: Comandos e funções

■ Manutenção BD: Backup's

■ Backup Logs

- **Descrição** : Captura todas as transações que ocorreram desde o último **backup de log** ou do último backup **COMPLETO**
- **Vantagens**: podem ser realizados muito frequentemente com impacto mínimo no desempenho. Restaura transação a transação.
- **Desvantagens**: Depende de toda uma cadeia de backup para a restauração. Se a cadeia “quebrar”, todo o **backup é perdido**.
 - Completo às 08:00h (1)
 - Backup log as 9:00h (2)
 - Backup log as 10:00h (3)
 - Backup log as 18:00h (4)

Restaurar:

(1) --> (2) --> (3) -->(4) -->.. (10)

(1) --> (2) --> ~~(3)~~ -->(4) -->.. (10)

Banco de Dados II: Comandos e funções

Manutenção BD: Backup's – **Implementação**

- backup database <nomeBase> to disk= <caminho>
- backup database <nomeBase> to disk= <caminho>
with DIFFERENTIAL
- backup LOG <nomeBase> to disk=
<caminho>

Banco de Dados II: Comandos e funções

Manutenção BD: Backup's – **Implementação**

■ Para ver as informações de um arquivo de backup

- RESTORE FILELISTONLY FROM DISK = <caminho do arquivo backup>

■ Restaurar backup

- RESTORE DATABASE <nome da base> FROM DISK = <caminho do arquivo de backup>

■ Restaurar backup com um nome / caminho diferente

- RESTORE DATABASE <nome da base> FROM DISK = 'caminho arquivo backup> WITH MOVE <nome_atual> TO <novo nome>, MOVE <Nome atual log> TO <novo nome LOG> RECOVERY

Banco de Dados II: Comandos e funções

Manutenção BD: Backup's – **Implementação**

- backup database teste to disk= 'f:\lixo\bkp_teste_b.bkp'
with ENCRYPTION
(ALGORITHM = AES_256,
SERVER CERTIFICATE = Certificadoservidor)

Banco de Dados II: Store Procedure

- Store Procedure

Banco de Dados II: Store Procedure

- Store Procedure

- Ou “procedimentos armazenados” são instruções pré-compiladas que ficam armazenados no banco de dados e podem serem “chamadas” a qualquer momento.
- Oferecem várias vantagens
 - Rapidez: As Store procedures são armazenadas pré-compilados no servidor, e são pré-carregadas junto com o Sql Server.
 - É possível alterar procedimentos de uma store procedure sem necessariamente alterar a aplicação que a usa.
 - Diminuição do fluxo na rede: Como a aplicação envia somente os parâmetros necessários para a Store procedure, todo o processamento é feito no servidor, que por sua vez devolve somente o resultado da operação, ou seja, o tráfego de informações na rede é mínimo.

Banco de Dados II: Store Procedure

- Store Procedure

- **CREATE Proc** <nome do procedimento>
(<parâmetros>
AS
<corpo do procedimento>;

- **Create proc Data**
as

```
select 'Hoje é ' + convert(varchar(12),getdate(),104) + ' ' + datename(dw,getdate())
```

Para executar:
Exec data

Banco de Dados II: Store Procedure

- **Store Procedure**

- As Store Procedures ficam armazenadas no próprio banco de dados, passando a fazer parte do mesmo. Se o banco de dados for transferido para outro servidor as SP também serão.
- Para EXCLUIR uma Store procedure deve-se usar o comando DROP PROC
 - Drop Proc data
- Para alterar uma Store Procedure deve-se usar o comando ALTER PROC

- **Alter Proc** <nome do procedimento>
(<parâmetros>)
as
<corpo do procedimento>

- **Alter Proc** proc Data
as
select 'Hoje é ' + **convert**(**varchar**(12),**getdate**(),104)+' '
+**datename**(dw,**getdate**())

Banco de Dados II: Store Procedure

- Store Procedure

Criar uma SP para efetuar uma rotina de backup semanal. A mesma deve ter os seguintes pontos:

- Deve criar um backup com um nome diferente para cada dia da semana. O nome do backup deve ser o nome do banco + o dia da semana:
 - Uvv_segunda.bkp
 - Uvv_terça.bkp
- O backup deve ser salvo na pasta 'c:\devc'
- Utilize o backup full

Banco de Dados II: View

- Views

Banco de Dados II: View

VIEWS

- É uma tabela única derivada de outra tabela, que pode ser uma tabela básica ou uma visão previamente definida;
- É considerada uma **tabela virtual**, uma vez que não existe de forma física;
- Isso limita as operações de atualização possíveis para as visões, embora não imponha nenhuma limitação para as consultas;

Banco de Dados II: View

VIEWS

- Em Sql server, a sintese para criação de uma view é:
 - `CREATE VIEW <view_name>`
`AS`
`<instrução_SELECT>`

Exemplo:

```
create view Atrasos  
AS
```

```
select pag_descricao, pag_valor, pag_dtvencimento,  
datediff(dd, pag_dtvencimento, getdate()) as Dias_atraso  
from pagar  
where pag_dtvencimento < getdate()  
and pag_dtpagto is null
```

Banco de Dados II: View

VIEWS

- A view ATRASOS irá criar uma tabela virtual com todas as faturas em atraso. O usuário pode usar essa tabela como uma tabela normal:
 - `Select * from atrasos`
ou
 - `Select * from atrasos`
`where Dias_atraso>30`

Banco de Dados II: View

VIEWS

- A view não é executada no momento de sua criação, mas **quando especificamos uma consulta sobre ela**. É responsabilidade do SGBD, e não do usuário, ter a certeza de que uma visão está atualizada

Banco de Dados II: View

VIEWS

- É possível alterar, incluir e excluir registros em uma view, desde que:
 - As definições de chaves primarias, e campos obrigatórios seguem mantidas
 - Não é possível atualizar campos derivados.
 - Não é possível atualizar views originadas de comandos de agregação(utilizando o group by)
 - As alterações feitas em uma view são replicadas para a tabela original, e vice versa.

Banco de Dados II: View

VIEWS – Cuidado!!

■ Delete view atrasos

- Apagaria TODOS os registros da view e os registros da tabela pagar que estavam na view.

■ DROP view atrasos

- Excluiria a view, sem afetar os registros da tabela pagar.

Banco de Dados II: View

VIEWS – Cuidado!!

- Nem todas as views são atualizáveis.
 - Views que incluem agregações, funções de conjunto ou junções complexas geralmente não podem ser atualizadas diretamente.
- Views regulares não possuem seus próprios índices.
 - Consultas sobre essas views podem não se beneficiar de índices tão eficientemente.
- View com funções de agregação acabam gerando uma sobrecarga, pois é atualizada na inserção/remoção de cada registro
- Manter índices em views indexadas (indexed views) pode aumentar a complexidade e sobrecarga de manutenção.

Banco de Dados II: View

VIEWS – Cuidado!!

- Requisitos para Criar uma View Indexada:
 - A view deve ser definida com a cláusula WITH SCHEMABINDING.
 - A view não pode incluir outras views.
 - A view não pode usar a cláusula DISTINCT, SUM, MIN, MAX, COUNT, TOP, OUTER JOIN, UNION, EXCEPT, ou INTERSECT.
 - A view não pode conter subconsultas, valores constantes ou funções agregadas que não sejam determinísticas (como por exemplo Getdate(), Rand())

Banco de Dados II: Trigger

- Triggers (Gatilhos)

Banco de Dados II: Trigger

Gatilhos (Triggers)

- Um Trigger é bloco de comandos SQL que é **automaticamente** executado quando um comando INSERT , DELETE ou UPDATE for executado em uma tabela do banco de dados.
- Os Triggers são usados para realizar tarefas relacionadas com validações , restrições de acesso , rotinas de segurança e consistência de dados ;
- Na verdade, pode ser visto como uma store procedure que é **disparada automaticamente após alguma ação específica em um tabela.**

Banco de Dados II: Trigger

Gatilhos (Triggers)

- `CREATE TRIGGER` nome_da_trigger
ON NOMETABLE
FOR
{ INSERT , UPDATE, DELETE }

Exemplo:

- `create trigger` testePag
on pagar
for insert
as
`select` 'foi inserido um registro no contas pagar'

Banco de Dados II: Trigger

Gatilhos (Triggers)

- Durante a execução das Triggers o SQL monta automaticamente:
 - A tabela INSERTED
 - A tabela DELETED;
- Dependendo do evento, estas tabelas irão conter:
 - INSERT:
 - *inserted* – linhas inseridas;
 - *deleted* – sem efeito;
 - UPDATE:
 - *inserted* – novo valor das linhas;
 - *deleted* – valores **ANTES** do update;
 - DELETE:
 - *inserted* – sem efeito;
 - *deleted* – linhas removidas;

Banco de Dados II: Trigger

Gatilhos (Triggers)

- Após a execução de uma ação com efeito sobre múltiplas linhas, as tabelas *inserted* e *deleted* conterão todas as linhas afetadas;
- A ação a ser realizada pelo trigger deve considerar (na maioria dos casos) todas as linhas afetadas;
- As tabelas *inserted* e *deleted*, embora sejam temporárias e residam em memória, podem ser consultadas através do comando SELECT (**mas somente dentro da própria trigger**);
- A variável global @@ROWCOUNT possui o número de linhas afetadas pela ação;

Banco de Dados II: Trigger

Gatilhos (Triggers)

- Create Trigger FaturasExcluidas

On pagar

For delete

As

declare @cont int

select @cont = count(*) from deleted

select 'Você excluiu ' + cast(@cont as
varchar(3)) + ' linha(s)'

Banco de Dados II: Trigger

Gatilhos (Triggers)

- Create Trigger FaturasExcluidas

On pagar

For delete

As

insert into log (tabela, nr_fatura, descricao,
data)

(select 'pagar', pag_fatura,
pag_descricao, getdate() from deleted)

Banco de Dados II: Trigger

Outros Tipos de Triggers

- NÃO é possível utilizar algumas operações dentro de uma trigger, tais como;
 - Alterar (alter) tabelas, procedures, trigger, indices etc..
 - Excluir (drop) tabelas, indices, procedures, trigger etc.,
 - Criar (create) tabela , bancos, indices, procedures, trigger etc...

**** Estas operações podem serem efetuadas através de Triggers de Banco de dados e Triggers de Servidor**

Banco de Dados II: Trigger

Outros Tipos de Triggers

- **Triggers DataBase:** Estes triggers são definidos dentro de um **banco de dados** específico e reagem a eventos de DDL que afetam os objetos desse banco.
 - São disparados através dos eventos CREATE, ALTER e DROP aplicados a qualquer **objeto estrutural do banco de dados**, tais como tabelas, visões, procedures, funções...

```
Create trigger logestrutura ON DATABASE FOR  
create_table, alter_procedure, drop_index  
AS  
begin  
end
```

Banco de Dados II: Trigger

Outros Tipos de Triggers

- **Triggers SERVIDOR:** Estes triggers são definidos para toda a **instância do servidor** e são utilizados para responder a eventos que têm um impacto mais amplo, afetando **múltiplos bancos de dados** ou a configuração do servidor como um todo.
 - São disparados através dos eventos CREATE, ALTER e DROP aplicados a **objetos que afetam o servidor**, tais como login, regras, configuração etc...

Create trigger logsrv ON ALL SERVER FOR
Create_login, Alter_server_configuration, logon

As...

Banco de Dados II: Trigger

Cuidado ao usar triggers:

- **Performance:** Triggers de tabela podem reduzir significativamente a performance do sistema, especialmente se eles contêm consultas complexas ou operações de longa duração.
- **Complexidade de Manutenção:** À medida que o sistema evolui, manter triggers pode se tornar desafiador, especialmente se houver muitos deles e se eles estiverem fazendo mudanças complexas nos dados.
- **Causa de Bugs e Erros:** Muitas vezes, os desenvolvedores podem esquecer que um trigger está em vigor, levando a resultados inesperados.

Banco de Dados II: Trigger

Cuidado ao usar triggers:

- **Performance:** Triggers de tabela podem reduzir significativamente a performance do sistema, especialmente se eles contêm consultas complexas ou operações de longa duração.
- **Complexidade de Manutenção:** À medida que o sistema evolui, manter triggers pode se tornar desafiador, especialmente se houver muitos deles e se eles estiverem fazendo mudanças complexas nos dados.
- **Causa de Bugs e Erros:** Muitas vezes, os desenvolvedores podem esquecer que um trigger está em vigor, levando a resultados inesperados.

BD II: Tratamento de erros

■ Tratamento de erros

BD II: Tratamento de erros

- **ROLLBACK TRANSACTION:** Em transações, o uso de ROLLBACK é essencial para garantir que, em caso de falha, as operações sejam revertidas para manter a consistência dos dados.
 - Em conjunto com BEGIN TRANSACTION, é possível iniciar uma transação, e caso ocorra um erro durante a execução, a transação é revertida com o ROLLBACK.
- **Variável de erro (@@ERROR em SQL Server):** o SQL Server, possui variáveis de erro, como @@ERROR, que armazena o código do erro ocorrido na última instrução.
 - Esse método pode ser usado para verificar o sucesso de uma operação e, em caso de erro, executar procedimentos de tratamento específicos.

BD II: Tratamento de erros

- **CUIDADO: Nem todos os erros disparam o @@Error.**
Geralmente, erros mais graves, não relacionados com comandos sql não disparam.
 - Erros do Sistema operacional
 - Erros de Interrupção de Sessão
 - Erros de Estouro de Stack / timeout
- O @@ERROR no SQL Server é atualizado **após cada comando executado**, portanto, ele precisa ser verificado **imediatamente** após a execução de cada instrução SQL, pois qualquer comando subsequente redefine o valor de @@ERROR

BD II: Tratamento de erros

```
alter proc transferencia
(
    @idbancoorigem    int,
    @idbancodestino  int,
    @valor            numeric(18,2)
) as
Begin tran
    update banco set ban_saldo = ban_saldo - @valor
    where idbanco=@idbancoorigem

    Update banco set ban_saldo = ban_saldo + @valor
    where idbanco=@idbancoDestino

    if @@error>0
        rollback
    else
        commit
return
```

BD II: Tratamento de erros

- **TRY...CATCH:** Muitos SGBDs, como SQL Server, oferecem a estrutura TRY...CATCH para capturar e manipular erros. Dentro
 - O bloco TRY é executado, e se ocorrer um erro, a execução passa para o bloco CATCH, onde é possível tratar o erro, registrar logs, ou executar comandos compensatórios.

BEGIN TRY

instrução

instrução

instrução.....

END TRY

Begin CATCH

instrução

instrução

instrução.....

END CATCH

BD II: Tratamento de erros

```
BEGIN TRY    --- inicia o bloco try
    UPDATE banco
    SET ban_saldo = ban_saldo - @valor
    WHERE idbanco = @idbancoorigem;

    UPDATE banco
    SET ban_saldo = ban_saldo + @valor
    WHERE idbanco = @idbancodestino;
end try
Begin catch
    --- se acontecer qualquer erro no bloco ty,
    --- o fluxo passa para o catch
    select 'Ops, deu erro!. O erro foi ' + ERROR_MESSAGE()
end catch
```

BD II: Tratamento de erros

- **THROW:** É usado para gerar um erro personalizado e interromper a execução de um bloco de código.

THROW Número do erro, 'Mensagem de erro', Estado

- **Número do erro:** O código de erro que você deseja lançar (precisa ser maior que 50000).
 - **Mensagem de erro:** O texto que será exibido.
 - **Estado:** Estado do erro (normalmente 1).
-
- **THROW : interrompe o processamento e “pula” para o catch**

BD II: Segurança

■ Segurança

BD II: Segurança

- Segurança em banco de dados é uma área bastante ampla que se refere a muitas questões, incluindo:
 - **Questões legais e éticas:** Trata dos direitos de acesso a informações específicas, garantindo que dados pessoais, financeiros e sensíveis sejam acessados apenas por aqueles que possuem autorização legal e ética para tal. Isso envolve conformidade com regulamentações como a **LGPD** (Lei Geral de Proteção de Dados).
 - Questões políticas nos níveis governamental, institucional ou corporativo (quais os tipos de informação que não devem ser tornadas disponíveis publicamente);
 - As necessidades em algumas organizações de identificar múltiplos níveis de segurança e em categorizar os dados e usuários com base nessas classificações. Exemplo: altamente secreto, secreto, confidencial e não confidencial;

BD II: Segurança

- **Confidencialidade, Integridade, Disponibilidade** são os principais pilares da segurança em bancos de dados e são amplamente conhecidos como o modelo **CIA** (Confidentiality, Integrity, Availability).
 - **Confidencialidade:** refere-se à proteção dos dados contra a divulgação não autorizada
 - **Integridade:** a informação deve ser protegida contra modificações impróprias (intencionais ou acidentais);
 - **Disponibilidade:** o acesso à informação deve estar disponível para um usuário ou programa que tenha direito legítimo a esta informação;

BD II: Segurança

Como garantir os pilares básicos de segurança (CIA)?

■ Proteção da **Confidencialidade**

- **Criptografia de Dados:** **Criptografar** dados armazenados e em trânsito para impedir que informações sensíveis sejam acessadas sem autorização.
- **Controle de Acesso Baseado em Funções:** Definir e aplicar políticas de acesso que permitam apenas o nível mínimo de acesso necessário para cada usuário.
- **Autenticação Forte:** Adote autenticação multifatorial e políticas rigorosas de senha para prevenir acessos não autorizados.

BD II: Segurança

Como garantir os pilares básicos de segurança (CIA)?

■ Proteção da **Confidencialidade**

● Criptografia a **nível de BANCO**.

- A criptografia a nível de banco criptografa os arquivos de dados e os arquivos de log do banco de dados, tornando os dados ilegíveis se os arquivos forem acessados fora do SQL Server (ou de outro servidor)
- A criptografia é gerenciada por uma **chave de criptografia do banco de dados (DEK)**, que é armazenada no próprio banco de dados. A DEK é protegida por um certificado armazenado no servidor
- O TDE é chamado de "transparente" porque **não altera a forma como consultas e operações** são executadas no SQL Server. Os dados são descriptografados automaticamente em tempo real quando acessados por usuários e aplicativos autorizados, e criptografados ao serem salvos em disco, sem que o usuário precise fazer nada..

BD II: Segurança

Como garantir os pilares básicos de segurança (CIA)?

- Proteção da **Confidencialidade**

- Criptografia a **nível de COLUNA**.

```
-- Abre a chave simétrica antes de usá-la
OPEN SYMMETRIC KEY MinhaChaveSimetrica DECRYPTION BY CERTIFICATE MeuCertificado;

-- Inserção de dados criptografados
INSERT INTO Clientes (Nome, CPF)
VALUES ('João', EncryptByKey(Key_GUID('MinhaChaveSimetrica'), '12345678901'));

-- Fecha a chave após o uso
CLOSE SYMMETRIC KEY MinhaChaveSimetrica;
```

BD II: Segurança

Como garantir os pilares básicos de segurança (CIA)?

■ Proteção da **Integridade**

- Controle de Validação de Entrada: Validar as entradas para **evitar injeções SQL** e garantir que os dados estejam no formato esperado antes de serem armazenados.
- Controle de Acesso Granular: Implementar permissões adequadas para que apenas usuários autorizados possam modificar dados, reduzindo o risco de alterações acidentais ou maliciosas.
- Uso de Triggers e Constraints: Utilizar restrições de integridade e triggers para evitar modificações indevidas nos dados, assegurando que informações incorretas não sejam inseridas ou alteradas.
- Auditorias e Logs de Atividade: Monitorar todas as atividades no banco de dados, registrando alterações e acessos para detectar e investigar rapidamente alterações inadequadas ou suspeitas.

BD II: Segurança

Como garantir os pilares básicos de segurança (CIA)?

■ Proteção da **Disponibilidade**

- **Backup e Recuperação de Dados:** Realize backups regulares e mantenha um plano de recuperação de desastres para restaurar rapidamente o acesso aos dados em caso de falhas ou ataques.
- **Defesas Contra Ransomware(*) e DoS(*):** Implemente soluções de monitoramento e detecção de ameaças para identificar atividades suspeitas, e use sistemas de defesa contra ataques de negação de serviço (DoS).
- **Redundância e Alta Disponibilidade:** Configure réplicas e failover automático para garantir que o banco de dados permaneça acessível, mesmo em caso de falhas no sistema ou manutenção.
- **Controle de Acesso à Rede:** Restrinja o acesso à rede para impedir acessos externos não autorizados, limitando a exposição do banco de dados.

BD II: Segurança

Como garantir os pilares básicos de segurança (CIA)?

■ Proteção da **Disponibilidade**

- **Ransomware** : é um tipo de malware que criptografa dados e exige um resgate (geralmente em criptomoeda) para restaurar o acesso.
- **DoS (Denial of Service)** é um ataque que visa sobrecarregar um sistema ou servidor, tornando-o incapaz de responder a requisições legítimas. No SQL Server, um ataque DoS pode ocorrer de várias formas:
 - **Sobrecarga de Requisições**: Um invasor pode enviar um grande número de consultas SQL complexas ou pesadas para sobrecarregar o servidor, exaurindo a CPU, a memória e o armazenamento temporário.
 - **Bloqueio de Sessões**: O invasor pode abrir várias sessões no SQL Server e mantê-las ativas, impedindo que usuários legítimos possam acessar o banco.

BD II: Segurança

- **DBA** (Administrador de Banco de Dados): É a autoridade máxima no gerenciamento do sistema de banco de dados, responsável por definir e manter a política de segurança e controle de acesso.
 - O DBA possui uma conta especial com permissões para realizar ações administrativas avançadas, como criação de contas e atribuição de privilégios.
 - **sa** no Sql server , **SYS** no oracle , **root** no mySql
- Os privilégios do DBA incluem comandos para conceder e revogar direitos para contas individuais de usuários ou de grupos de usuários, e comandos para a realização das seguintes ações do tipo:
 - Criação de Contas;
 - Concessão de privilégio discricionários;
 - Revogação de privilégio;
 - Atribuição de nível de segurança;

BD II: Segurança

- Criação de Contas: O sql server possui vários “níveis” de autenticação.
 - **Login de Usuário:** é a identidade no nível do servidor e representa um usuário que possui permissões para se conectar ao **servidor SQL**.
 - O login permite que o usuário se **autentique no servidor**, mas **não concede** automaticamente permissões dentro de um **banco de dados específico**.
 - **Autenticação do Windows:** Utiliza contas ou grupos de usuários do Windows para login.
 - **Autenticação do SQL Server:** Cria logins com credenciais específicas para o SQL Server, independentes do sistema operacional.

BD II: Segurança

- Criação de Contas: O sql server possui vários “níveis” de autenticação.

- **Autenticação do Windows:**

`CREATE LOGIN [EMPRESA\lucas] FROM WINDOWS;`

- **Autenticação do SQL Server:**

`CREATE LOGIN lucas with password = '123'`

Importante: O ‘login’ fornece **acesso somente ao servidor de banco de dados**. Os bancos de dados ainda estão inacessíveis.

BD II: Segurança

- Criação de Contas: O sql server possui vários “níveis” de autenticação.
- **Conta de Usuário:** O usuário do banco de dados é uma entidade de segurança criada **dentro de um banco de dados específico**.
 - Um ‘**USUÁRIO**’ **NÃO** representa uma pessoa, e sim **OQUE** um determinado login pode fazer
 - No SQL Server, um “usuário” do banco de dados geralmente é associado a um **login de servidor**.
 - O login autentica o usuário no servidor SQL,
 - O “usuário” define o que o usuário pode fazer dentro do banco específico.

BD II: Segurança

- Criação de Contas: O sql server possui vários “níveis” de autenticação.
- **Conta de Usuário:** O usuário do banco de dados é uma entidade de segurança criada dentro de um banco de dados **específico**.
 - **CREATE USER** usu_todos **FOR LOGIN** lucas
 - **CREATE USER** user_consulta **FOR LOGIN** maria
 - **CREATE USER** user_gravacao **FOR LOGIN** jose;

Após a criação, deve-se definir oque cada conta de usuário poderá acessar. Pode-se usar o comando GRANT ou grupo de regras (Roles)

BD II: Segurança

- Criação de Contas: O sql server possui vários “níveis” de autenticação.
- **Conta de Usuário:** O usuário do banco de dados é uma entidade de segurança criada dentro de um banco de dados **específico**.
 - **GRANT SELECT** ON **Pagar** TO user_consulta ;
 - **GRANT INSERT** ON PAGAR TO user_gravacao ;
 - **GRANT SELECT** ON dbo.pagar (idpagar,fkempresa TO user_consulta
 - **DENY** SELECT ON dbo.pagar (idpagar,fkempresa TO user_consulta
 - **Revoke** SELECT ON dbo.pagar (idpagar,fkempresa) TO user_consulta
 - **GRANT CONTROL ON DATABASE::uvw** TO todos;
 - **GRANT EXECUTE TO** usu_todos ; // qualquer SP
 - **GRANT EXECUTE ON OBJECT::"SP_iu_pagar"** TO usu_todos ;

BD II: Segurança

- Criação de Contas: O sql server possui vários “níveis” de autenticação.
- **ROLES:** é um conjunto de permissões que podem ser atribuídas a usuários ou logins. Roles permitem gerenciar permissões de forma centralizada, agrupando-as e simplificando a administração de segurança.
 - **Roles de Servidor:** Definem permissões no nível do servidor SQL, **aplicáveis a todos os bancos de dados**. Exemplos incluem sysadmin e serveradmin.
 - **Roles de Banco de Dados:** Definem permissões dentro de um banco de dados específico. Exemplos incluem db_datareader (permite leitura de dados), db_datawriter (permite escrita de dados) e db_owner (controle total do banco de dados).

BD II: Segurança

- Criação de Contas: O sql server possui vários “níveis” de autenticação.
- **ROLES:** Funcionam como um GRUPO com determinados direitos, onde você pode acionar usuários

```
ALTER ROLE db_datawriter ADD MEMBER usu_todos;
```

```
ALTER ROLE db_datareader ADD MEMBER usu_consulta;
```


BD II: Segurança

Tipo	Role	Descrição
Servidor	sysadmin	Controle total sobre o servidor SQL.
	serveradmin	Gerencia configurações de servidor.
	securityadmin	Gerencia logins e permissões de segurança.
	dbcreator	Cria, altera, restaura e remove bancos de dados.
Banco de Dados	db_owner	Controle total sobre o banco de dados.
	db_datareader	Permite leitura em todas as tabelas e views.
	db_datawriter	Permite escrita (inserir, atualizar, excluir) em todas as tabelas e views.
	db_backupoperator	Executa backups de banco de dados.
	db_ddladmin	Executa comandos de definição de dados (DDL) no banco de dados.
	db_denydatareader	Nega permissão de leitura em todas as tabelas e views.
	db_denydatawriter	Nega permissão de escrita em todas as tabelas e views.

BD II: Segurança

- **ROLES:** É possível criar ROLES personalizáveis

```
USE uvw
```

```
CREATE ROLE Role_financeira;
```

```
Go
```

```
GRANT SELECT, INSERT, UPDATE, DELETE ON pagar TO Role_financeira;
```

```
GRANT SELECT, INSERT, UPDATE, DELETE ON receber TO Role_financeira;
```

--inserir usuário na roles

```
ALTER Role_financeira ADD usu_financeiro
```

Banco de Dados II: Segurança

■ TRANSAÇÕES

Banco de Dados II: Transações

■ TRANSAÇÕES

Dados Relacionais II : Transações

- Sistemas Monousuários x Multiusuários
 - Critério de classificação quanto ao número de usuários que podem usar o sistemas **Concorrentemente**, ou seja, ao mesmo tempo;
 - SGBD Monousuário – um usuário de cada vez pode usar o sistema;
 - SGBD Multiusuário – muitos usuários podem usar o sistema, acessando o banco de dados concorrentemente. Este é o tipo mais largamente utilizado hoje;
 - Pelo fato de múltiplos usuários acessarem os dados ao mesmo tempo, exigem mecanismos sofisticados para gerenciar a **concorrência** e garantir a integridade dos dados.

Dados Relacionais II : Transações

■ Sistemas Monousuários x Multiusuários

- Por padrão, todo banco de dados criado no SQL server é Multiusuário, porém é possível mudar isso através do comando:
 - **ALTER DATABASE** <nome da base> **SET** <tipo <desejado>
- **Alter database** basepitagoras **set** multi_user
 - Define a base como multiusuário. Este é o padrão
- **Alter database** basepitagoras **set** single_user
 - Define a base como Mono usuário, ou seja, somente um usuário pode utilizar o banco de dados de cada vez.
- **Alter database** basepitagoras **set** Restricted_user
 - Define que somente usuários administradores e quem criou o banco terão acesso ao mesmo.

Dados Relacionais II : Transações

- **TRANSAÇÃO**: É uma *sequencia de operações executada em um banco de dados* que acessa e eventualmente modifica dados;
 - Uma transação começa com o banco de dados consistente, porém, **DURANTE** a execução da transação o banco de dados pode ficar temporariamente inconsistente;
 - Quando a transação é finalizada o Banco de Dados deve ficar consistente;
 - Problemas a considerar no tratamento de transações:
 - Falhas
 - Execução concorrente de varias transações

Dados Relacionais II : Transações

- **TRANSAÇÃO**: Transações devem obedecer às propriedades **ACID**:
 - **Atomicidade**: A propriedade de atomicidade garante que as transações sejam atômicas (indivisíveis). A transação será executada totalmente ou não será executada.
 - **Consistência**: A propriedade de consistência garante que o banco de dados passará de uma forma consistente para outra forma consistente.
 - **Isolamento**: A propriedade de isolamento garante que a transação não sofrerá interferência de nenhuma outra transação concorrente.
 - **Durabilidade**: A propriedade de durabilidade garante que o que foi salvo, não será mais perdido.

Dados Relacionais II : Transações

- Existem vários casos em que é necessário definir um BLOCO de instruções (varias transações), onde o bloco todo deve ser executado com sucesso, ou nenhum dos comandos dos blocos.
 - Exemplo: Transferir um valor de uma conta para outras

```
alter proc transferencia
(
    @idbancoorigem    int,
    @idbancodestino  int,
    @valor            numeric(18,2)
) as
update banco set ban_saldo = ban_saldo - @valor
where idbanco=@idbancoorigem

Update banco set ban_saldo = ban_saldo + @valor
where idbanco=@idbancoDestino

return
```

- E se desse problema no segundo Update??

Dados Relacionais II : Transações

- Para garantir que um bloco seja TODO executado com sucesso, ou nenhum comando do bloco seja executado, o SQL permite estabelecer explicitamente declarações de **início de transação** e **fim de transação**;
- Isso é feito através dos comandos:
 - BEGIN TRAN : Define o início de uma transação.
 - Commit : Define o fim de uma transação
 - Rollback : Cancela uma transação

Dados Relacionais II : Transações

- Ao utilizar BEGIN TRAN antes de um bloco, as alterações feitas no banco somente serão gravadas quando for efetuado um COMMIT.

```
alter proc transferencia
(
    @idbancoorigem    int,
    @idbanco destino  int,
    @valor            numeric(18,2)
) as
Begin tran
    update banco set ban_saldo = ban_saldo - @valor
    where idbanco=@idbancoorigem

    Update banco set ban_saldo = ban_saldo + @valor
    where idbanco=@idbancoDestino

    if @@error>0
        rollback
    else
        commit
return
```

Dados Relacionais II : Transações

■ Exemplo

Begin Tran

Delete pagar

Select * from pagar

Obs: ao executar os comandos acima você verá que TODOS os registros da tabela pagar foram excluídos.

Agora digite **Rollback** e execute somente este comando.

Você verá que os registro da tabela PAGAR REAPARECERAM

Dados Relacionais II : Transações

- **@@error**: Captura o código de erro após a execução de uma instrução T-SQL que resulta em erro:
 - **CUIDADO: Nem todos os erros disparam o @@Error.** Geralmente, erros mais graves, não relacionados com comandos sql não disparam.
 - Erros do Sistema operacional
 - Erros de Interrupção de Sessão
 - Erros de Estouro de Stack / timeout

Estes erros devem ser tratados com TRY / CATH

Dados Relacionais II : Transações

- Estrutura Try/ catch :

- BEGIN TRY

.....

END TRY

BEGIN CATCH

....

END CATCH

- ERROR_MESSAGE()

Dados Relacionais II : Transações

- **Concorrência** : Uma concorrência ocorre sempre que duas ou mais transações tentam acessar o mesmo dado, ao mesmo tempo, podendo gerar informações inconsistentes.
 - Verificar se possui o produto em estoque
 - Inserir um novo registro da venda
 - Debitar o estoque
 - Thread **A** verifica o estoque (passo #1) e insere o registro da venda (passo #2)
 - thread **A** é bloqueada e **B** passa a ser executada
 - Thread **B** verifica o estoque (passo #1), o qual ainda não foi debitado, e insere o registro da venda (passo #2)
 - Thread **B** atualiza o estoque (passo #3), que agora fica zerado
 - Thread **B** é bloqueada e **A** passa a ser executada
 - Thread **A** atualiza o estoque (passo #3), que agora fica **negativo**!

Dados Relacionais II : Concorrência

- **Concorrência** : Principais efeitos gerados pela concorrência.
 - **Leitura Suja (Dirty Read)** : Ocorre sempre que um usuário lê algum dado que ainda não foi comitado, e a operação é desfeita com o rollback.
 - **Leitura Suja** geralmente configura ou gera um ERRO.

Dados Relacionais II : Concorrência

- **Concorrência** : Principais efeitos gerados pela concorrência.
 - **Leitura Suja (Dirty Read)** :
 - **Transação A** começa e atualiza o saldo de uma conta bancária de \$1000 para \$900.
 - **Transação B** lê o saldo da conta enquanto a Transação A ainda está em andamento e vê o saldo como \$900.
 - **Transação A** é revertida, voltando o saldo para \$1000.
 - **Transação B** agora trabalhou com informações que nunca foram confirmadas, o que pode levar a decisões baseadas em dados incorretos.

Dados Relacionais II : Concorrência

- **Atualizações perdidas** : Uma atualização perdida ocorre quando duas transações, que acessam o mesmo dado, atualizam este dado de forma concorrente, resultando em uma das atualizações sendo sobrescrita pela outra.
- Isso geralmente acontece quando uma transação não bloqueia adequadamente os dados que está modificando.
- **Configuram um ERRO**: Isso geralmente acontece quando uma transação não bloqueia adequadamente os dados que está modificando.

Dados Relacionais II : Concorrência

■ Atualizações perdidas :

- **Transação A** lê o saldo de uma conta bancária que é \$1000.
- **Transação B** também lê o mesmo saldo de \$1000, simultaneamente.
- **Transação A** efetua um credito de R\$ 100,00 atualizando o saldo para \$1100.
- **Transação B**, efetua um DEBITO de 100,00 atualizando o saldo para \$900 sem saber da alteração feita pela Transação A.
- O resultado final é que a atualização feita pela Transação A é perdida, pois o saldo final é de apenas \$900 em vez de \$1000.

Dados Relacionais II : Concorrência

- **Leitura não repetíveis** : Acontece quando a mesma transação acessa uma linha várias vezes e lê dados diferentes a cada vez.



Dados Relacionais II : Concorrência

- **Leitura Não repetível** não é exatamente um erro, pois ela sempre apresenta o status do banco em um determinado instante.

Dados Relacionais II : Concorrência

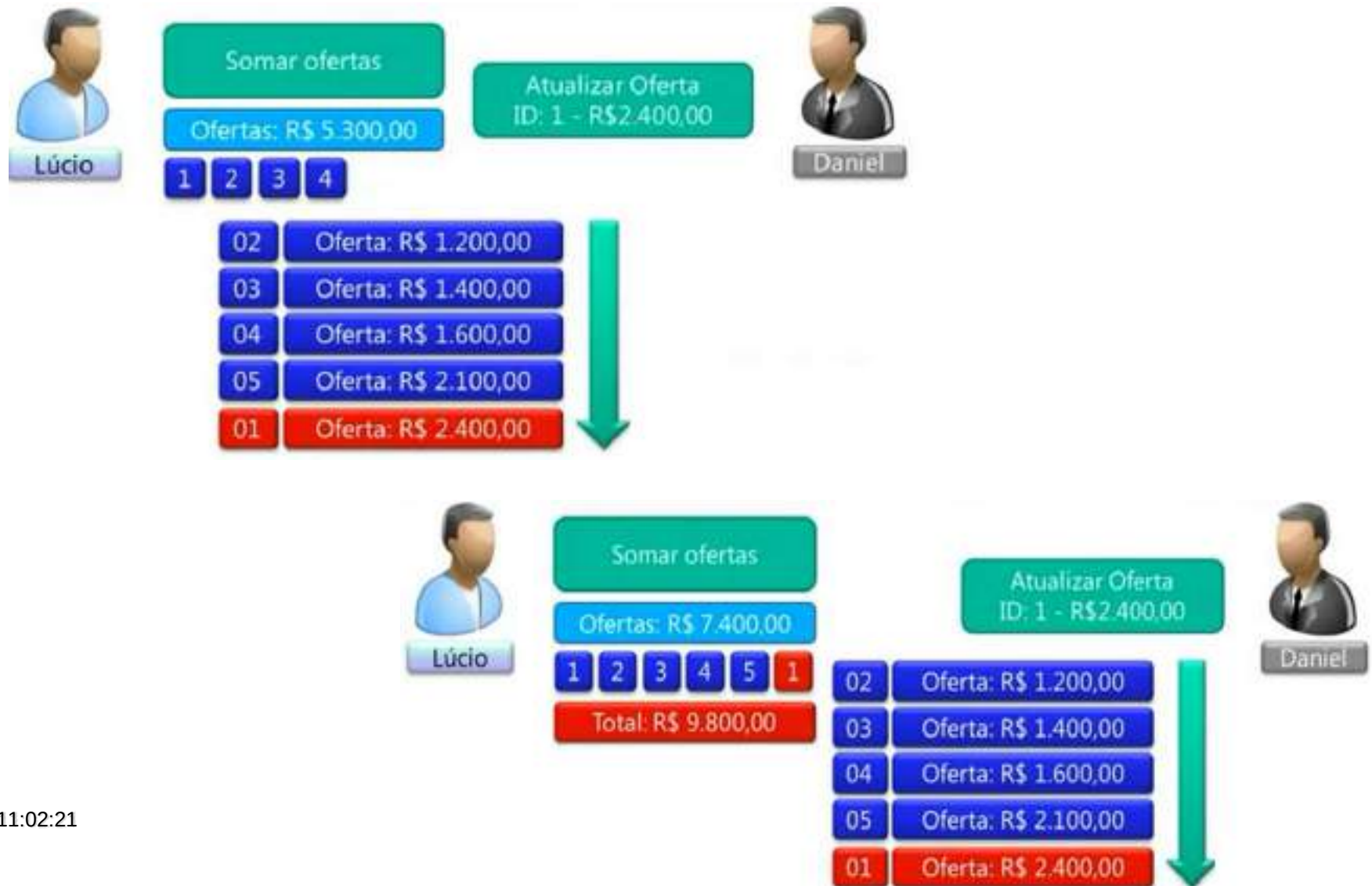
- **Registros Fantasmas:** É a mesma idéia da leitura não repetível, a diferença, é que registros fantasmas são NOVOS registros INCLUÍDOS após a leitura/totalização da transação.
- No exemplo anterior, se o usuário Daniel INCLUIR um registro com valor de R\$ 5.500,00 esse registro irá para o topo da lista. Se o usuário Lucio já tiver lido os primeiros registros, a totalização ignorar o novo registro inserido.

Dados Relacionais II : Concorrência

- **Leitura Duplas:** Acontece quando a mesma linha é lida mais de uma vez na mesma transação.



Dados Relacionais II : Concorrência



Dados Relacionais II : Transações

- Sistemas Monousuários x Multiusuários
 - Critério de classificação quanto ao número de usuários que podem usar o sistemas **Concorrentemente**, ou seja, ao mesmo tempo;
 - SGBD Monousuário – um usuário de cada vez pode usar o sistema;
 - SGBD Multiusuário – muitos usuários podem usar o sistema, acessando o banco de dados concorrentemente. Este é o tipo mais largamente utilizado hoje;
 - Pelo fato de múltiplos usuários acessarem os dados ao mesmo tempo, exigem mecanismos sofisticados para gerenciar a **concorrência** e garantir a integridade dos dados.

Dados Relacionais II : Transações

■ Sistemas Monousuários x Multiusuários

- Por padrão, todo banco de dados criado no SQL server é Multiusuário, porém é possível mudar isso através do comando:

- **ALTER DATABASE** <nome da base> **SET** <tipo <desejado>

- **Alter database** basepitagoras **set** multi_user
 - Define a base como multiusuário. Este é o padrão
- **Alter database** basepitagoras **set** single_user
 - Define a base como Mono usuário, ou seja, somente um usuário pode utilizar o banco de dados de cada vez.
- **Alter database** basepitagoras **set** Restricted_user
 - Define que somente usuários administradores e quem criou o banco terão acesso ao mesmo.

Dados Relacionais II : Transações

- **TRANSAÇÃO**: É uma *sequencia de operações executada em um banco de dados* que acessa e eventualmente modifica dados;
 - Uma transação começa com o banco de dados consistente, porém, **DURANTE** a execução da transação o banco de dados pode ficar temporariamente inconsistente;
 - Quando a transação é finalizada o Banco de Dados deve ficar consistente;
 - Problemas a considerar no tratamento de transações:
 - Falhas
 - Execução concorrente de varias transações

Dados Relacionais II : Transações

- **TRANSAÇÃO**: Transações devem obedecer às propriedades **ACID**:
 - **Atomicidade**: A propriedade de atomicidade garante que as transações sejam atômicas (indivisíveis). A transação será executada totalmente ou não será executada.
 - **Consistência**: A propriedade de consistência garante que o banco de dados passará de uma forma consistente para outra forma consistente.
 - **Isolamento**: A propriedade de isolamento garante que a transação não sofrerá interferência de nenhuma outra transação concorrente.
 - **Durabilidade**: A propriedade de durabilidade garante que o que foi salvo, não será mais perdido.

Dados Relacionais II : Transações

- Existem vários casos em que é necessário definir um BLOCO de instruções (varias transações), onde o bloco todo deve ser executado com sucesso, ou nenhum dos comandos dos blocos.
 - Exemplo: Transferir um valor de uma conta para outras

```
alter proc transferencia
(
    @idbancoorigem    int,
    @idbancodestino   int,
    @valor             numeric(18,2)
) as
update banco set ban_saldo = ban_saldo - @valor
where idbanco=@idbancoorigem

Update banco set ban_saldo = ban_saldo + @valor
where idbanco=@idbancodestino

return
```

- E se desse problema no segundo Update??

Dados Relacionais II : Transações

- Para garantir que um bloco seja TODO executado com sucesso, ou nenhum comando do bloco seja executado, o SQL permite estabelecer explicitamente declarações de **início de transação** e **fim de transação**;
- Isso é feito através dos comandos:
 - BEGIN TRAN : Define o início de uma transação.
 - Commit : Define o fim de uma transação
 - Rollback : Cancela uma transação

Dados Relacionais II : Transações

- Ao utilizar BEGIN TRAN antes de um bloco, as alterações feitas no banco somente serão gravadas quando for efetuado um COMMIT.

```
alter proc transferencia
(
    @idbancoorigem    int,
    @idbanco destino  int,
    @valor            numeric(18,2)
) as
Begin tran
    update banco set ban_saldo = ban_saldo - @valor
    where idbanco=@idbancoorigem

    Update banco set ban_saldo = ban_saldo + @valor
    where idbanco=@idbancoDestino

    if @@error>0
        rollback
    else
        commit
    return
```


Dados Relacionais II : Transações

■ Exemplo

Begin Tran

Delete pagar

Select * from pagar

Obs: ao executar os comandos acima você verá que TODOS os registros da tabela pagar foram excluídos.

Agora digite **Rollback** e execute somente este comando.

Você verá que os registro da tabela PAGAR REAPARECERAM

Dados Relacionais II : Transações

- **Concorrência** : Uma concorrência ocorre sempre que duas ou mais transações tentam acessar o mesmo dado, ao mesmo tempo, podendo gerar informações inconsistentes.
 - Verificar se possui o produto em estoque
 - Inserir um novo registro da venda
 - Debitar o estoque
 - Thread **A** verifica o estoque (passo #1) e insere o registro da venda (passo #2)
 - thread **A** é bloqueada e **B** passa a ser executada
 - Thread **B** verifica o estoque (passo #1), o qual ainda não foi debitado, e insere o registro da venda (passo #2)
 - Thread **B** atualiza o estoque (passo #3), que agora fica zerado
 - Thread **B** é bloqueada e **A** passa a ser executada
 - Thread **A** atualiza o estoque (passo #3), que agora fica **negativo**!

Dados Relacionais II : Concorrência

- **Concorrência** : Principais efeitos gerados pela concorrência.
 - **Leitura Suja (Dirty Read)** : Ocorre sempre que um usuário lê algum dado que ainda não foi comitado, e a operação é desfeita com o rollback.
 - **Leitura Suja** geralmente configura ou gera um ERRO.

Dados Relacionais II : Concorrência

- **Concorrência** : Principais efeitos gerados pela concorrência.
 - **Leitura Suja (Dirty Read)** :
 - **Transação A** começa e atualiza o saldo de uma conta bancária de \$1000 para \$900.
 - **Transação B** lê o saldo da conta enquanto a Transação A ainda está em andamento e vê o saldo como \$900.
 - **Transação A** é revertida, voltando o saldo para \$1000.
 - **Transação B** agora trabalhou com informações que nunca foram confirmadas, o que pode levar a decisões baseadas em dados incorretos.

Dados Relacionais II : Concorrência

- **Atualizações perdidas** : Uma atualização perdida ocorre quando duas transações, que acessam o mesmo dado, atualizam este dado de forma concorrente, resultando em uma das atualizações sendo sobrescrita pela outra.
- Isso geralmente acontece quando uma transação não bloqueia adequadamente os dados que está modificando.
- **Configuram um ERRO**: Isso geralmente acontece quando uma transação não bloqueia adequadamente os dados que está modificando.

Dados Relacionais II : Concorrência

■ Atualizações perdidas :

- **Transação A** lê o saldo de uma conta bancária que é \$1000.
- **Transação B** também lê o mesmo saldo de \$1000, simultaneamente.
- **Transação A** efetua um credito de R\$ 100,00 atualizando o saldo para \$1100.
- **Transação B**, efetua um DEBITO de 100,00 atualizando o saldo para \$900 sem saber da alteração feita pela Transação A.
- O resultado final é que a atualização feita pela Transação A é perdida, pois o saldo final é de apenas \$900 em vez de \$1000.

Dados Relacionais II : Concorrência

- **Leitura não repetíveis** : Acontece quando a mesma transação acessa uma linha várias vezes e lê dados diferentes a cada vez.



Dados Relacionais II : Concorrência

- **Leitura Não repetível** não é exatamente um erro, pois ela sempre apresenta o status do banco em um determinado instante.

Dados Relacionais II : Concorrência

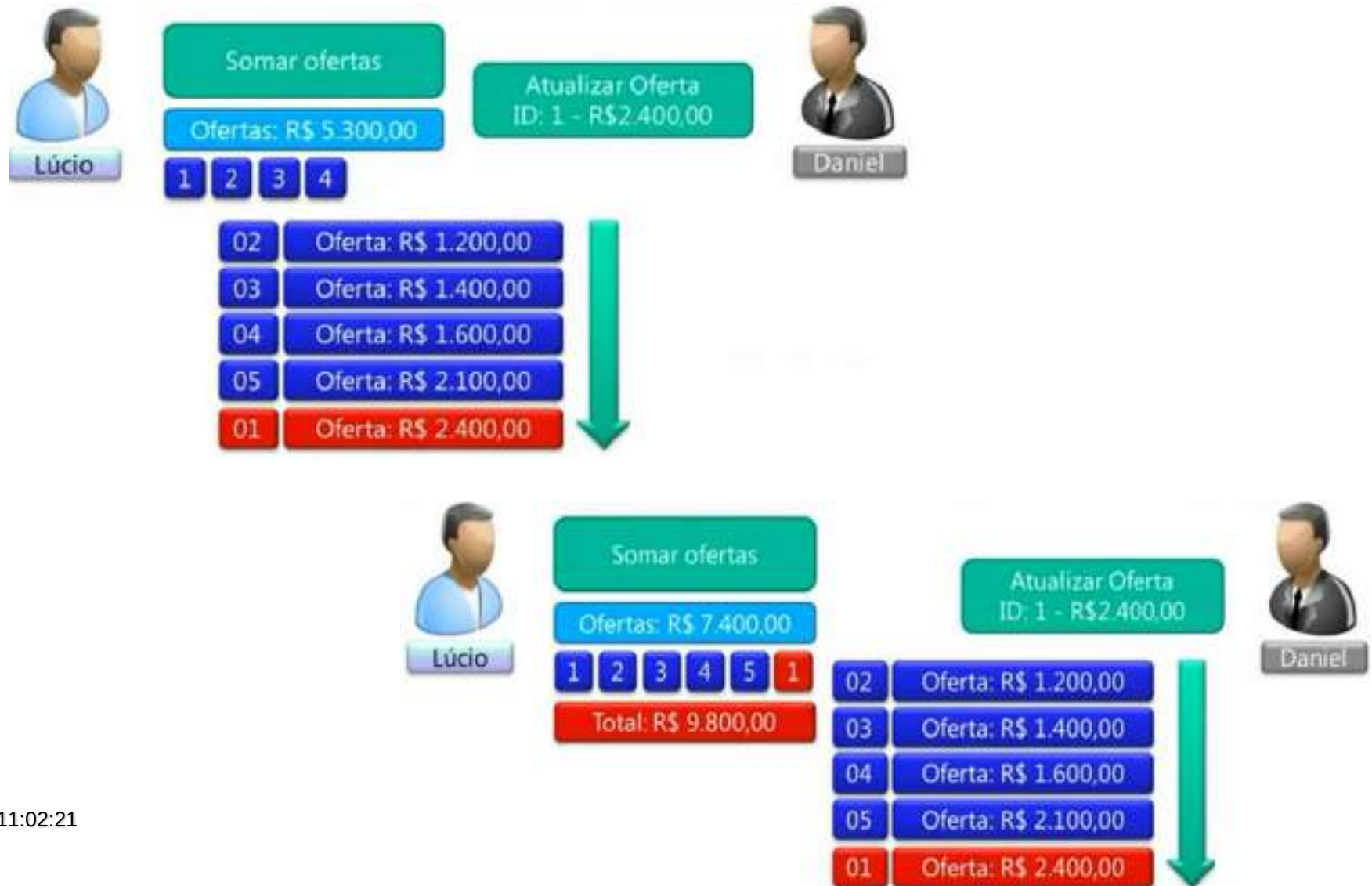
- **Registros Fantasmas:** É a mesma idéia da leitura não repetível, a diferença, é que registros fantasmas são NOVOS registros INCLUÍDOS após a leitura/totalização da transação.
- No exemplo anterior, se o usuário Daniel INCLUIR um registro com valor de R\$ 5.500,00 esse registro irá para o topo da lista. Se o usuário Lucio já tiver lido os primeiros registros, a totalização ignorar o novo registro inserido.

Dados Relacionais II : Concorrência

- **Leitura Duplas:** Acontece quando a mesma linha é lida mais de uma vez na mesma transação.



Dados Relacionais II : Concorrência



Dados Relacionais II : Concorrência

- **Leituras Repetidas:** Causam ERROS de consistências.

Dados Relacionais II : Concorrência

- **Leituras perdidas:** Acontece quando um registro é deixado de ler em função de uma movimentação de dados.
 - Imagine o mesmo exemplo da leitura dupla, porém agora Daniel alterar o registro 5 para um valor de R4 900,00 . O registro irá passar para a primeira posição da lista, porém Lucio que estava lendo o registro 3 não irá ler este registro, pois já passou pelo mesmo.
 - Também causa um **ERRO de consistência**.

Dados Relacionais II : Concorrência

- **Modelos de concorrência:** São estruturas de gerenciamento que controlar as concorrências de forma a garantir a consistência do banco de dados
 - São formadas por uma série de normas e regras que tentam conciliar concorrência com a consistência do banco.
 - Existem três modelos
 - Chaos (Caos)
 - Pessimista
 - Otimista

Dados Relacionais II : Concorrência

Modelos de concorrência:

- **Modelo Chaos:** Não existe bloqueio de registro ou controle de atualização/leitura.
 - Existe uma ausência de regras/normas, um verdadeiro caos.
 - Ideal para sistemas monousuário
 - Nenhuma banco atual utiliza este tipo de modelo.

Dados Relacionais II : Concorrência

Modelos de concorrência:

- **Modelo Pessimista:** Pressupõem que haverá muitas atualizações concorrentes, ou seja, muitos usuários acessando o mesmo recurso/registro.
 - Impõem bloqueios aos registros lidos/escritos
 - Leituras bloqueiam escritas e vice versa
 - É o modo padrão do SQL server
 - Utiliza memória para o gerenciamento dos bloqueio. O bloqueio é feito através de tabelas de bloqueio.

Dados Relacionais II : Concorrência

Modelos de concorrência:

- **Modelo Otimista:** Pressupõem que haverá POUCAS atualizações concorrentes, ou seja, poucos usuários iram acessar o mesmo recurso/registo ao mesmo tempo.
 - Leituras NÃO bloqueiam escritas e vice versa
 - Usa sempre a última versão commitada do registo.
 - Quando duas pessoas acessarem o mesmo registo ao mesmo tempo, ocorrerá conflitos em potencial.

Dados Relacionais II : Níveis de isolamento

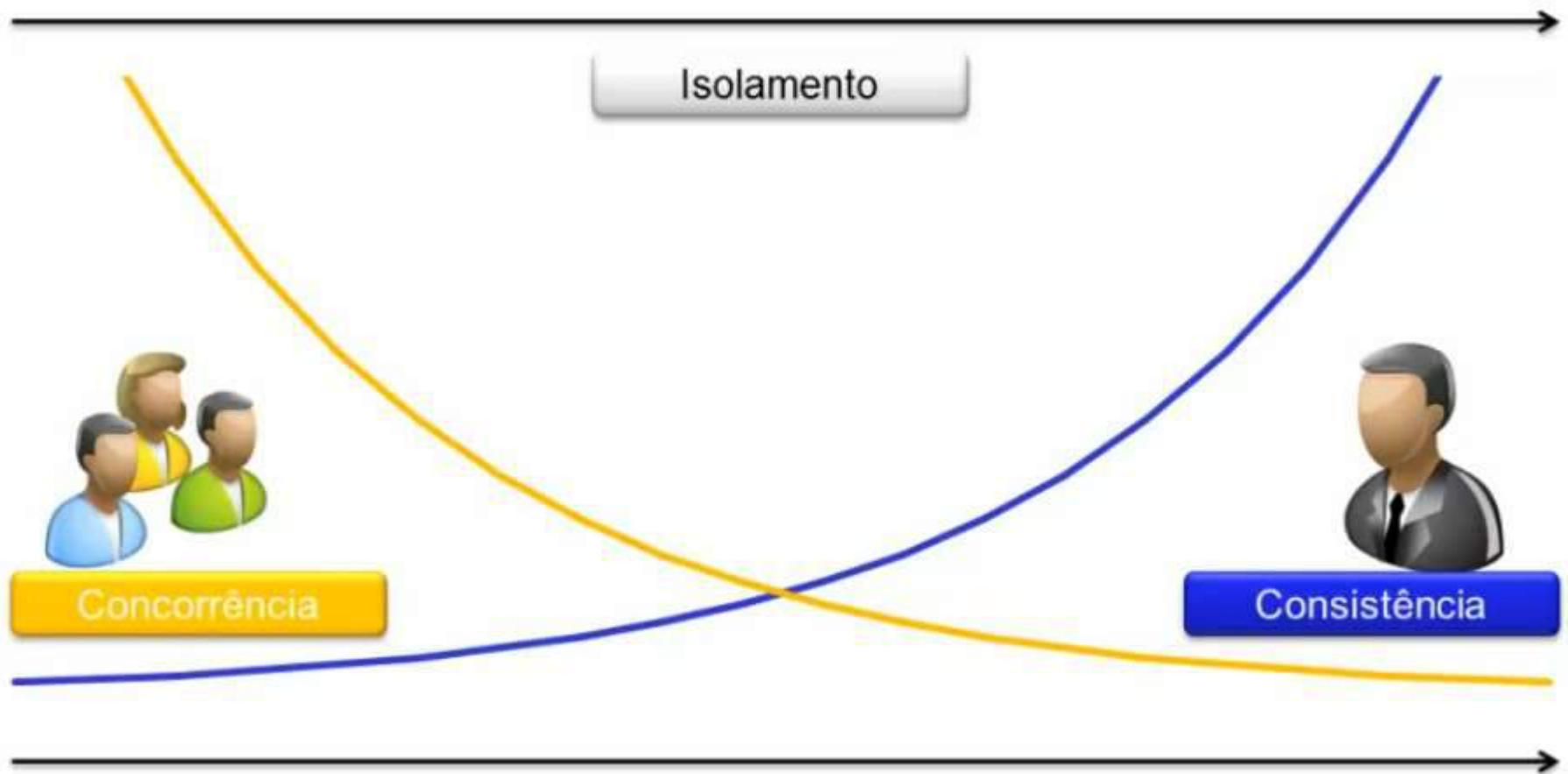
- **Nível de isolamento das transações:**
 - O nível de isolamento irá definir entre outras coisas:
 - Quando é obrigatório bloquear ou não um recurso
 - Duração do bloqueio
 - Tratamento de conflitos
 - Leituras em um registro devem bloquear escritas neste registro?
 - Escritas em um registro devem bloquear leituras?
 - As leituras devem ver os dados NÃO comitados?

Dados Relacionais II : Níveis de isolamento

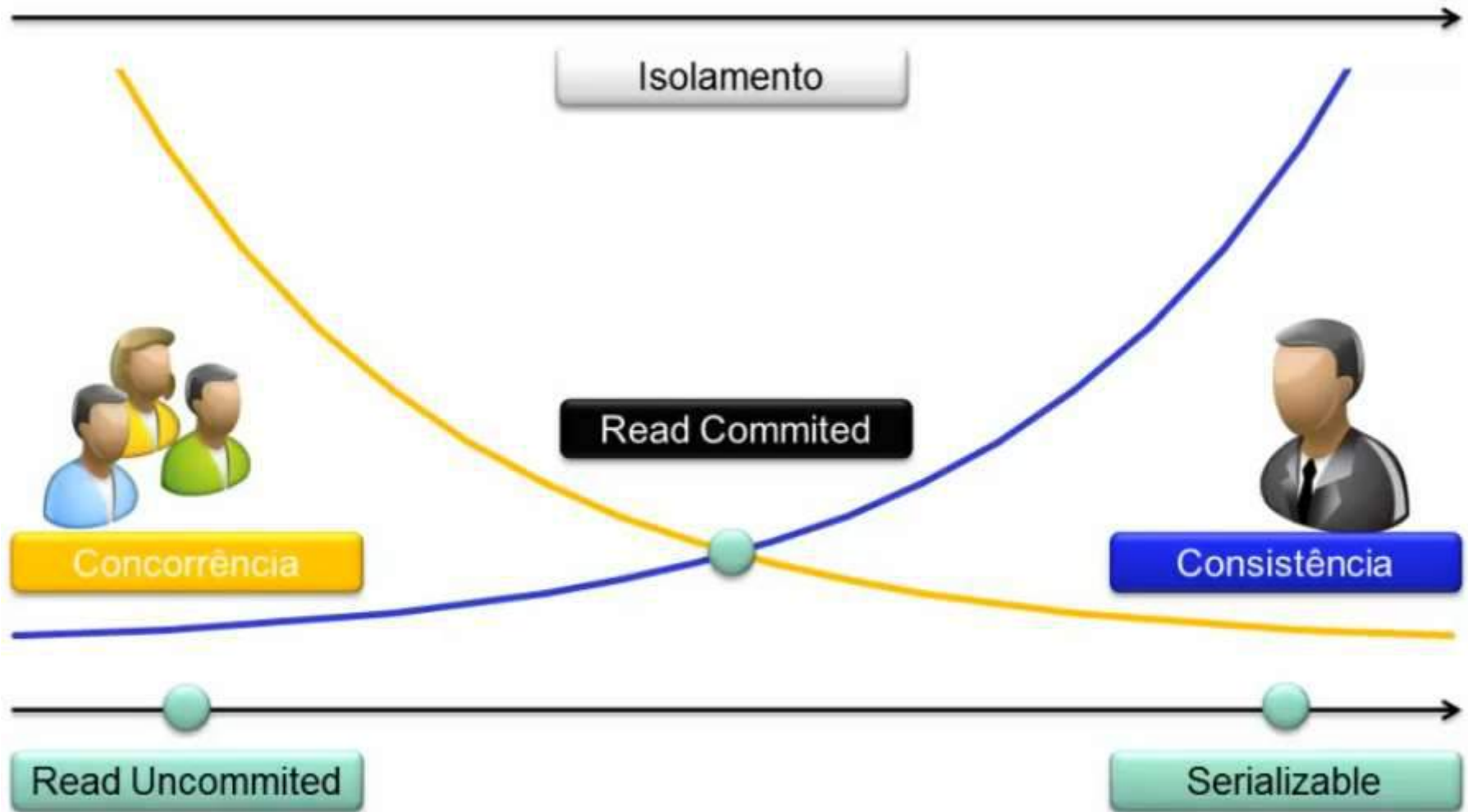
- **Nível de isolamento das transações:** O nível de isolamento define o grau de influência que uma transação pode ter sobre a outra.
 - Conforme os princípios ACID, deve existir Isolamento perfeito entre as transações, ou seja, uma transação não pode influenciar o resultado de outra.
 - Porém na prática isso significaria você serializa todas as transações, ou seja, colocar todas as transações em uma fila e executar as mesmas em ordem seqüencial.
 - Isso comprometeria a performance e o número de usuários simultâneos seria extremamente baixo.
 - Os níveis de isolamentos existem para isso, em alguns casos será necessário ter um isolamento perfeito entre as transações, enquanto que em outros, pode-se optar por um isolamento menos radical.

Dados Relacionais II : Níveis de isolamento

■ Nível de isolamento das transações:



Dados Relacionais II : Níveis de isolamento



Dados Relacionais II : Níveis de isolamento

■ Nível de isolamento das transações:

- **Repeatable read** : Este tipo de isolamento segura os registros para a conexão na leitura até o fim da transação, nenhum usuário pode alterar ou apagar estes registros até que a conexão encontre um commit ou rollback (implícito ou explícito), Os Dirty's Read's e NonRepeatable Read's não ocorrem. Só utilizar este tipo de isolamento quando estritamente necessário.
- **Seriazable** : Este é o tipo de isolamento mais restrito, este não permite a leitura (select), atualização (update), inserção (insert) ou remoção (delete) de nenhum registro que está sendo lido, até que encontre o commit ou rollback (implícito ou explícito). Os Dirty Read's, NonRepeatable Reads não ocorrem. Só utilizar este tipo de isolamento quando estritamente necessário.
- **READ COMMITTED** : Este é o isolation level default usado pelo SQL no Read Committed os registros são bloqueados quando se está fazendo a leitura do mesmo, por isto o tipo de inconsistência Dirty Read não existe, lê somente informações comitadas. Ao dar o update também o registro é bloqueado para a conexão até o commit ou rollback (implícito ou explícito).
- **READ UNCOMMITTED** : No Read Committed os registros AINDA NÃO comitados são mostrados normalmente. Não oferece isolamento

Dados Relacionais II : Níveis de isolamento

■ Nível de isolamento das transações:

● Definir o nível de isolamento

- SET TRANSACTION ISOLATION LEVEL

Exemplos:

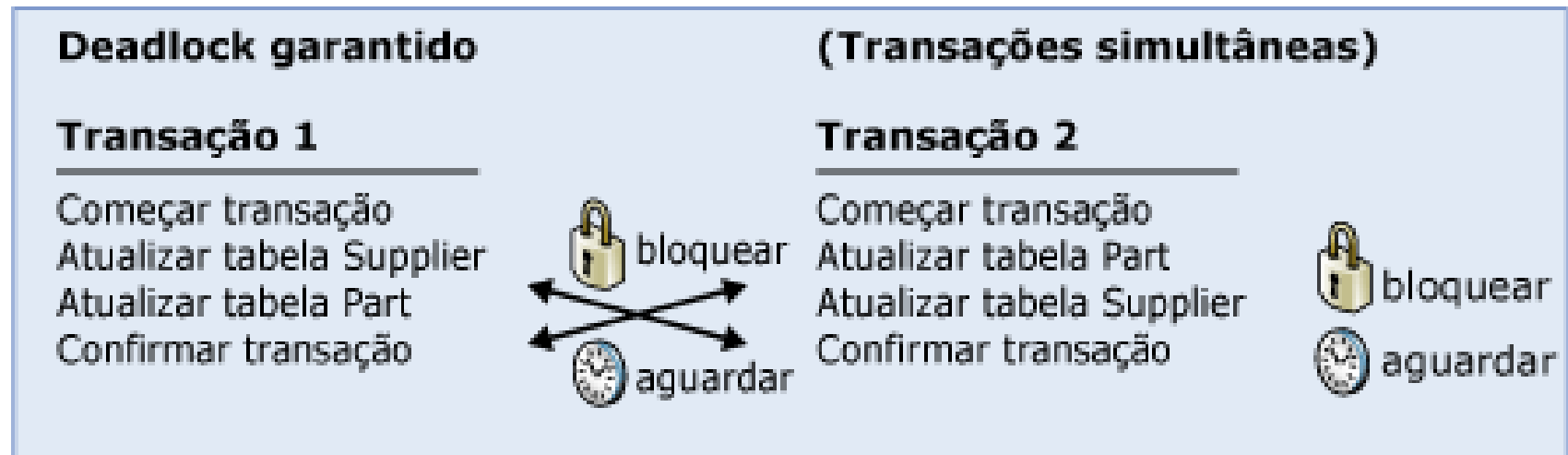
- SET TRANSACTION ISOLATION LEVEL READ COMMITTED
- SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED
- SET TRANSACTION ISOLATION LEVEL REPEATABLE READ
- SET TRANSACTION ISOLATION LEVEL SERIALIZABLE

● Consultar nível de isolamento

- DBCC USEROPTIONS

Dados Relacionais II : Níveis de isolamento

- **DEADLOCK:** É uma situação em que duas ou mais transações estão em estado simultâneo de espera, cada uma aguardando que uma das demais libere um bloqueio para ela poder prosseguir;



Dados Relacionais II : Níveis de isolamento

■ STARVATION

- ocorre se o algoritmo seleciona uma mesma transação como vítima repetidamente, causando abort's repetidos e nunca acabando a execução;